MICROSOFT CERTIFIED: FABRIC ANALYTICS ENGINEER ASSOCIATE

Exam DP-600 Study Guide

Implementing Analytics Solutions Using Microsoft Fabric

EXAM **DP-600**	SKILLS OUTLINE VERSION **19 October 2026**
TIME TO COMPLETE **100 minutes, proctored**	PASSING SCORE **700 of 1000**
LEVEL **Intermediate · Associate**	RENEWAL **Free online, every 12 months**

OneLake · Lakehouse · Warehouse · Dataflows Gen2 · T-SQL · KQL · DAX · Direct Lake · Semantic models · Git & deployment pipelines

Every Microsoft link in this guide carries the Microsoft Learn Student Ambassadors Contributor ID `wt.mc_id=studentamb_657938`. Language-locale segments have been removed and the ID is appended with `?` or `&` exactly as the Registered Member Handbook requires, so you can copy any link straight out of this document and share it.

Built from the official skills outline published on the Microsoft Learn DP-600 study guide page.

Table of contents

Chapter 0 — How to use this guide

0.1 What DP-600 actually certifies

0.2 The three skill areas and their weights

0.3 Exam logistics you should know before booking

0.4 A six-week study plan

0.5 How to read the callouts

Chapter 1 — The Microsoft Fabric platform in one chapter

1.1 What Fabric is

1.2 Tenant, capacity, workspace, item

1.3 OneLake and shortcuts

1.4 The medallion architecture

1.5 Choosing a data store

1.6 Items you must be able to recognise

Chapter 2 — Maintain a data analytics solution, part 1: security and governance

2.1 The mental model: every gate must open

2.2 Workspace-level access control

2.3 Item-level access control

2.4 File-level access control with OneLake security

2.5 Row-level security (RLS)

2.6 Column-level and object-level security

2.7 Sensitivity labels

2.8 Endorsement

2.9 Chapter 2 self-check

2.10 Learn more

Chapter 3 — Maintain a data analytics solution, part 2: the development lifecycle

3.1 Git integration

3.2 Power BI Desktop projects (.pbip)

3.3 Deployment pipelines

3.4 Impact analysis and lineage

3.5 The XMLA endpoint

3.6 Reusable assets

3.7 Chapter 3 self-check

3.8 Learn more

Chapter 4 — Prepare data, part 1: get data

4.1 Connections and gateways

4.2 Discovering data: OneLake catalog and Real-Time hub

4.3 Ways to get data in

4.4 OneLake integration for Eventhouse and semantic models

4.5 Chapter 4 self-check

4.6 Learn more

Chapter 5 — Prepare data, part 2: transform data

5.1 Choosing a transformation engine

5.2 Implementing a star schema

5.3 Denormalising, joining, and aggregating

5.4 Data quality: duplicates, nulls, types

5.5 Views, functions, and stored procedures

5.6 Chapter 5 self-check

5.7 Learn more

Chapter 6 — Prepare data, part 3: query and analyse

6.1 The Visual query editor

6.2 T-SQL essentials for the exam

6.3 KQL essentials for the exam

6.4 DAX essentials for the exam

6.5 The same question in three languages

6.6 Learn more

Chapter 7 — Implement and manage semantic models, part 1: design and build

7.1 Storage modes

7.2 Relationships

7.3 DAX the outline names explicitly

7.4 Calculation groups, dynamic format strings, field parameters

7.5 Large semantic model storage format

7.6 Composite models

7.7 Chapter 7 self-check

7.8 Learn more

Chapter 8 — Implement and manage semantic models, part 2: optimise at enterprise scale

8.1 Find the bottleneck before you fix it

8.2 Report and query-level improvements

8.3 DAX performance patterns

8.4 Direct Lake in depth

8.5 Direct Lake on OneLake vs Direct Lake on SQL analytics endpoint

8.6 Incremental refresh

8.7 Chapter 8 self-check

8.8 Learn more

Chapter 9 — The hands-on lab track

Chapter 10 — Practice questions

Chapter 11 — Exam-day checklist and quick reference

11.1 The week before

11.2 The 60-second memory dump

11.3 Reading exam questions

11.4 Glossary

11.5 Where to go next

Appendix A — One-page decision tables

A.1 Where should this data live?
A.2 Which ingestion tool?
A.3 Which security control?
A.4 Which lifecycle tool?

Chapter 0 — How to use this guide

0.1 What DP-600 actually certifies

Passing **Exam DP-600: Implementing Analytics Solutions Using Microsoft Fabric** earns the Microsoft Certified: Fabric Analytics Engineer Associate credential. Microsoft describes the role like this:

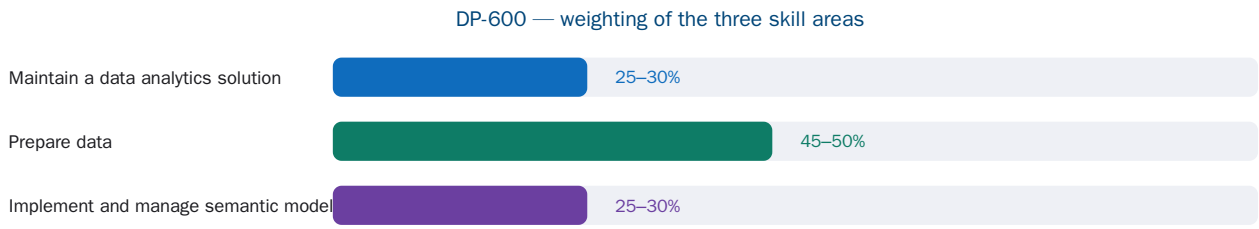
> As a candidate for this exam, you should have subject matter expertise in designing, creating, and managing analytical assets, such as semantic models, warehouses, or lakehouses.

Your three responsibilities, straight from the official DP-600 study guide:

- Prepare and enrich data for analysis
- Secure and maintain analytics assets
- Implement and manage semantic models

You are expected to query and analyse data with **SQL**, **KQL**, and **DAX**. You are *not* expected to be a Spark specialist — that is DP-700's territory — but you must be able to read a PySpark notebook and know when a notebook is the right tool.

0.2 The three skill areas and their weights



Skills measured as of 19 October 2026. Nearly half the exam is "Prepare data" — budget your study time accordingly.

Skill area	Weight	Chapters in this guide
Maintain a data analytics solution	25–30%	2, 3
Prepare data	45–50%	4, 5, 6
Implement and manage semantic models	25–30%	7, 8

> **KEY TAKEAWAY**
> "Prepare data" is worth almost as much as the other two areas combined. If you are short on time, make sure you can confidently get data in, transform it into a star schema, and query it in SQL, KQL, and DAX.

0.3 Exam logistics you should know before booking

Item	Detail
Duration	100 minutes to complete the assessment; the appointment itself is longer (see exam duration and experience)
Passing score	700 on a scale of 1000 — this is a scaled score, not a percentage. See exam scoring and score reports
Question count	Microsoft does not publish it. Expect roughly 40–60 items, typically including one or more case studies
Question types	Multiple choice, multi-select, drag-and-drop ordering, hot area, build-list, and possible interactive components
Languages	English, Japanese, Chinese (Simplified), German, French, Spanish, Portuguese (Brazil)
Delivery	Proctored, online or at a Pearson VUE test centre
Retake	24 hours after a first failure, then longer waits — see the exam retake policy
Renewal	Free online assessment on Microsoft Learn every 12 months — renew your certification
Accommodations	Extra time and assistive technology available — request accommodations

> **TIP**
> Take the free DP-600 practice assessment at least twice: once before you study to find your gaps, and once in the final week. Try the exam sandbox first so the question interface itself is never a surprise.

0.4 A six-week study plan



A realistic pace at roughly 6–8 hours per week. Compress to three weeks by doubling weekly hours, but never skip the hands-on labs.

Week	Focus	This guide	Microsoft Learn
1	Platform, OneLake, data stores	Ch. 1, 4	Get started with Microsoft Fabric
2	Lakehouse, warehouse, transformations	Ch. 5	Implement a Lakehouse · Implement a data warehouse
3	SQL, KQL, DAX querying	Ch. 6	Design and transform analytics data
4	Security, governance, ALM	Ch. 2, 3	Secure and govern analytics data · Manage a Fabric environment
5	Semantic models and Direct Lake	Ch. 7, 8	Design and manage semantic models
6	Practice, weak spots, exam readiness	Ch. 9, 10, 11	Practice assessment · Exam Readiness Zone

0.5 How to read the callouts

Callout	What it means
Exam focus	Wording drawn straight from the skills outline — expect to be tested here
Common exam trap	A distinction the exam deliberately tests; most wrong answers come from these
Know the difference	Two features that look alike; be able to choose between them under time pressure
Key takeaway	The one sentence to remember from the section
Hands-on lab	Do this in a real workspace. Reading alone will not get you to 700
Tip / **Watch out**	Practical advice and known gotchas

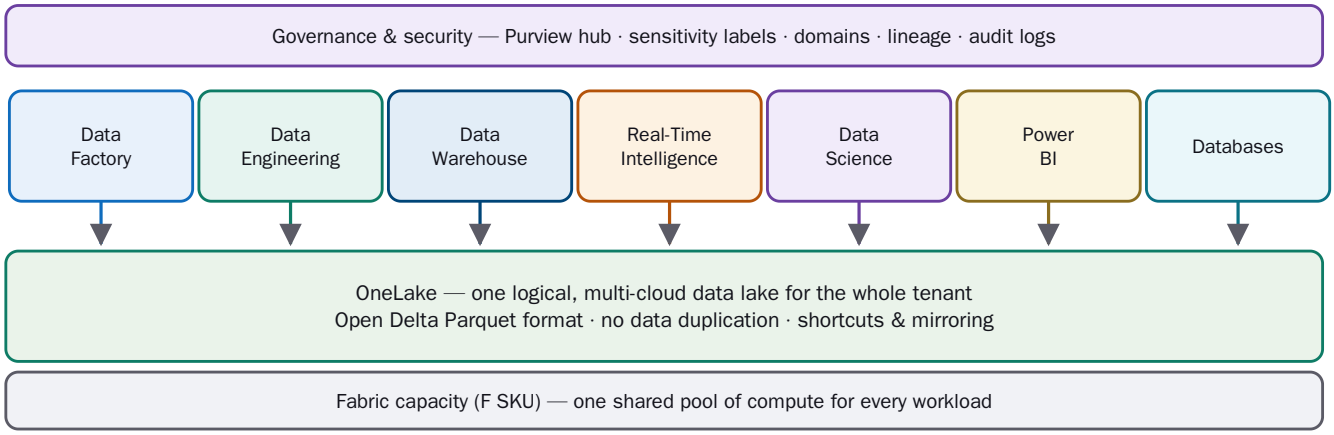
> **HANDS-ON LAB**
>
> Everything in this guide can be practised for free. Start a Microsoft Fabric trial (60 days, F64-equivalent trial capacity) or use the free capacity that comes with Power BI Premium Per User. Create one workspace named `dp600-lab` and build everything there.

Chapter 1 — The Microsoft Fabric platform in one chapter

Every other chapter assumes the vocabulary in this one. If you can explain the picture below out loud, you are ready to move on.

1.1 What Fabric is

Microsoft Fabric is a **SaaS analytics platform**. Instead of provisioning a separate service for storage, ETL, warehousing, streaming, and BI, you buy one pool of compute (a capacity) and get every workload on top of one storage layer (OneLake).



The Fabric stack: one capacity, one lake, many workloads, governed as a whole.

Three consequences matter for the exam:

- **One copy of the data.** All workloads persist to OneLake in open Delta Parquet. A table written by Spark is immediately queryable by T-SQL and by a Direct Lake semantic model — with no copy, no export, no pipeline.
- **One security and governance surface.** Sensitivity labels, lineage, endorsement, and domains apply to every item type, not per-service.
- **One billing surface.** Every workload consumes Capacity Units (CUs) from the same F SKU. A runaway Spark job can slow down someone else's report.

1.2 Tenant, capacity, workspace, item

Level	What it is	Who administers it
Tenant	Your whole Microsoft Entra organisation. Tenant settings switch features on or off globally	Fabric administrator, in the admin portal
Capacity	The compute resource, sold as an **F SKU** (F2 up to F2048). Each workspace is assigned to exactly one capacity	Capacity administrator, via capacity settings
Domain	A logical grouping of workspaces by business area (Finance, Sales) used for federated governance	Domain admin, see domains
Workspace	The container for items and the main unit of access control	Workspace Admin, see workspaces
Item	A lakehouse, warehouse, notebook, pipeline, semantic model, report, eventhouse, and so on	Item permissions, on top of workspace roles

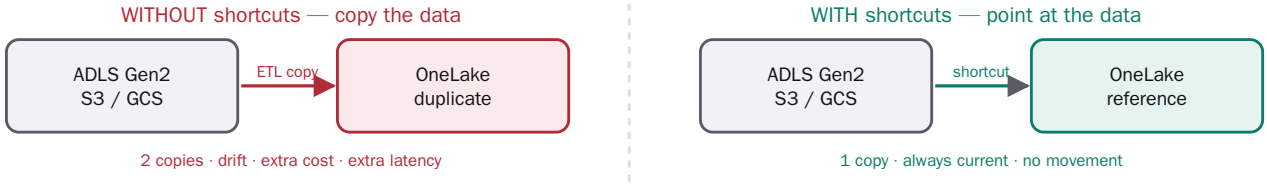
Capacity behaviour you should recognise:

- CU consumption is **smoothed** over time; sustained overuse leads to throttling, which delays interactive queries and background jobs differently.
- Capacities can be **paused and resumed** (pause and resume) — pausing stops billing but also stops everything running on it.
- You monitor consumption with the Microsoft Fabric Capacity Metrics app.
- You can scale a capacity up or down; some features (for example large semantic models and certain Direct Lake guardrails) depend on the SKU size.

1.3 OneLake and shortcuts

OneLake is a single logical data lake for the entire tenant, built on ADLS Gen2, automatically provisioned, and organised as `tenant / workspace / item / Files or Tables`. You cannot opt out of it, and you do not size it.

A **shortcut** is a pointer to data that lives somewhere else — another lakehouse, another workspace, ADLS Gen2, Amazon S3, Google Cloud Storage, Dataverse, or an on-premises location via a gateway. Reading through a shortcut looks exactly like reading a local folder.



A shortcut removes the copy, the copy job, and the drift between the two copies.

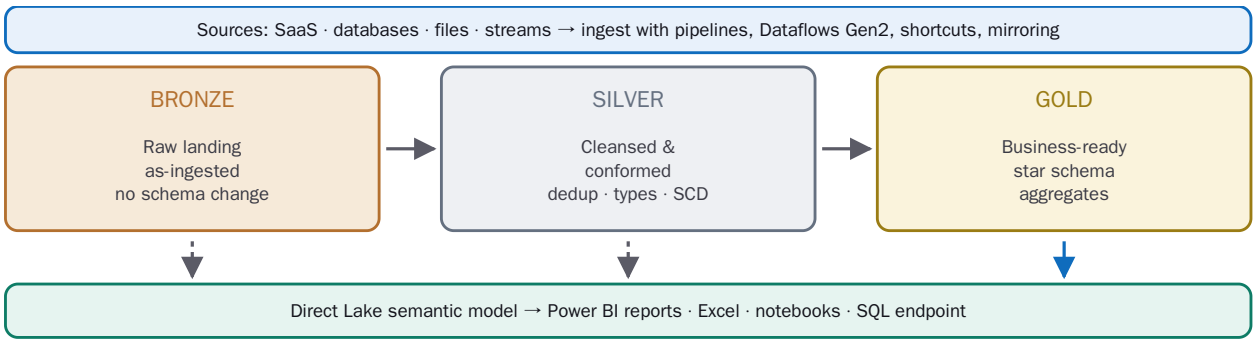
Concept	Data movement	Freshness	Typical use
Shortcut	None — read in place	Always current	Reuse a Gold table across workspaces; read S3/ADLS without ETL
Mirroring	Managed replication into OneLake	Near real time	Bring Azure SQL, Cosmos DB, Snowflake into Fabric with no ETL
Copy (pipeline/dataflow)	Physical copy	As of last run	You need to reshape, cleanse, or land history

COMMON EXAM TRAP

A shortcut does **not** carry the source's permissions into Fabric automatically. Access through a shortcut is evaluated by the shortcut's own connection and by OneLake security in the target workspace. Shortcuts to external clouds use the stored credential of the connection, not the caller's identity (unless the connection uses a pass-through identity). Expect a question that hinges on "who can see this data after I create the shortcut?"

1.4 The medallion architecture

Fabric does not force a layout, but every Microsoft reference architecture and most exam scenarios use medallion:



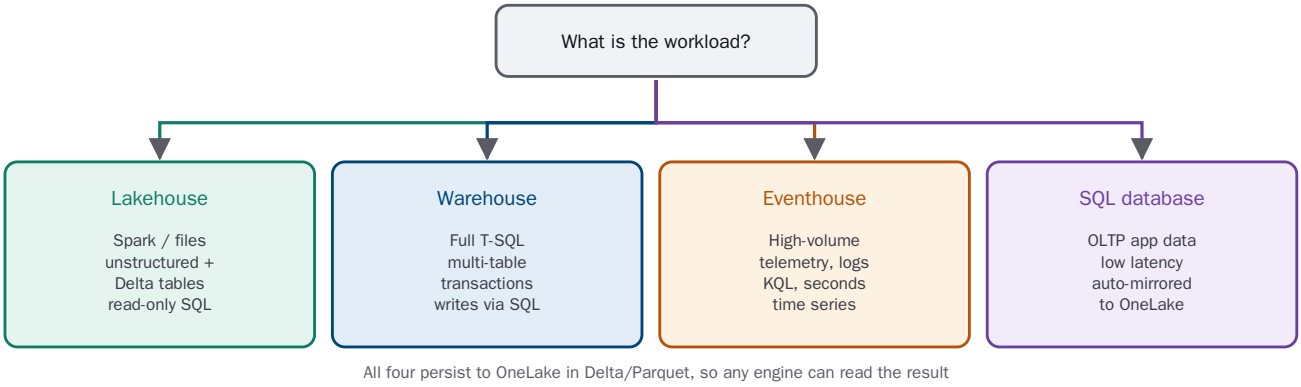
Bronze keeps the raw truth, Silver makes it correct, Gold makes it useful. Report from Gold.

- **Bronze** — land the source data as-is. No business logic. You can always replay from here.
- **Silver** — cleansed and conformed: types fixed, duplicates removed, late-arriving rows handled, slowly changing dimensions maintained.
- **Gold** — business-ready star schemas and aggregates, shaped for the semantic model.

Each layer can be a separate lakehouse, a separate schema, or a separate workspace. Separate workspaces give the cleanest security boundary; separate schemas in one lakehouse give the simplest development experience.

1.5 Choosing a data store

This is an explicit exam objective in both "Get data" and "Ingest and transform data", and it appears in case studies.



The four analytical stores. All of them land data in OneLake in Delta/Parquet.

	Lakehouse	Warehouse	Eventhouse (KQL)	SQL database
Primary language	PySpark, Spark SQL	T-SQL	KQL (T-SQL subset for reads)	T-SQL
Writes	Spark, pipelines, Dataflows Gen2	**T-SQL** `INSERT` / `UPDATE` / `DELETE` / `MERGE`, COPY INTO	Streaming ingestion, `.ingest`	Full OLTP transactions
Multi-table transactions	No	**Yes**	No	Yes
Unstructured files	**Yes** (Files area)	No	No	No
Best at	Data engineering, ML, mixed file types	Dimensional models, SQL-first teams, complex joins	Telemetry, logs, IoT, time series, sub-second over huge volumes	Operational apps, low-latency point reads
SQL endpoint	Read-only	Read/write	—	Read/write
Auto-mirrors to OneLake	Native	Native	Yes, via OneLake availability	Yes, automatically

KNOW THE DIFFERENCE

Lakehouse vs Warehouse is the single most common "choose a store" question. Decide with two questions: *(1) Does the team need to write with T-SQL, including multi-table transactions?* → Warehouse. *(2) Does the workload involve files, Spark, or ML?* → Lakehouse. If both, use both: a lakehouse for Bronze/Silver and a warehouse for Gold, joined through shortcuts or cross-database queries.

Full decision guidance: choose a data store and choose between pipeline, dataflow, and Spark.

1.6 Items you must be able to recognise

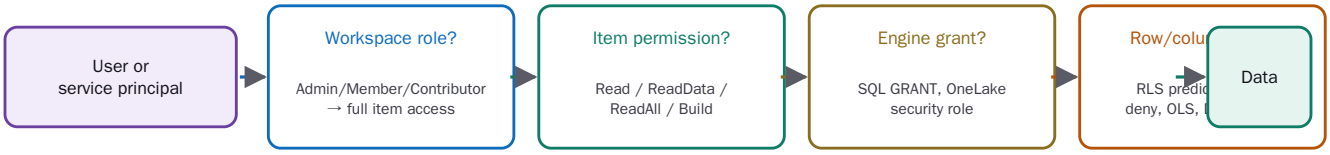
Item	One-line purpose
Lakehouse	Delta tables + file storage, with a read-only SQL analytics endpoint and a default semantic model
Warehouse	Full T-SQL relational warehouse, create a warehouse
Notebook	Spark code (PySpark, Spark SQL, Scala, R), how to use notebooks
Data pipeline	Orchestration and Copy activity, Data Factory in Fabric
Dataflow Gen2	Power Query transformations with a destination, Dataflows Gen2 overview
Eventstream	No-code streaming ingest and routing, Eventstream overview
Eventhouse	KQL databases for real-time data, Eventhouse
Semantic model	The tabular model that reports query with DAX
Report / dashboard	Power BI visuals and Real-Time Dashboards
Environment	Reusable Spark configuration and libraries, environments
Variable library	Values that change per deployment stage, variable library

> **KEY TAKEAWAY**
> One capacity, one lake, one security model. Every "where should this live?" question is answered by the language the team writes in and whether the data is files, relational, or streaming.

Chapter 2 — Maintain a data analytics solution, part 1: security and governance

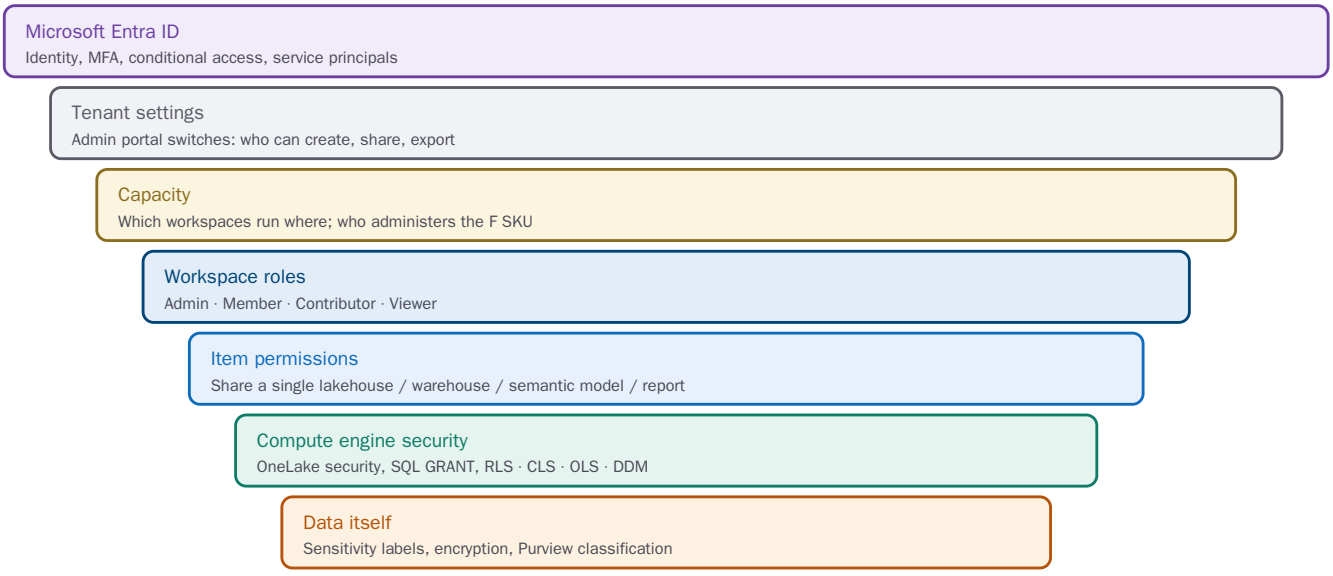
> **EXAM FOCUS**
>
> **Implement security and governance** — implement workspace-level access controls · implement item-level access controls · implement row-level, column-level, object-level, and file-level access control · apply sensitivity labels to items · endorse items.

2.1 The mental model: every gate must open



Every gate must allow the request; the most restrictive rule always wins

A request is authorised at four gates. The most restrictive rule always wins — there is no "allow" that overrides a "deny".



Broadest scope at the top — each layer narrows what the layer above allowed

The full stack of controls, from identity down to the data itself.

2.2 Workspace-level access control

A workspace role grants a blanket level of access to **every item** in the workspace. Assign roles to **Microsoft Entra security groups**, never to individuals — this is both best practice and a common correct answer.

Capability	Admin	Member	Contributor	Viewer
Delete the workspace, change its capacity	✔			
Add or remove people in any role	✔			
Add people as Contributor or Viewer	✔	✔		
Share items and re-share with others	✔	✔		
Publish and update an app	✔	✔		
Create, edit, delete items	✔	✔	✔	
Schedule and run refreshes and jobs	✔	✔	✔	
View and interact with items, read data through the SQL endpoint	✔	✔	✔	✔
Read raw files through Spark / OneLake APIs	✔	✔	✔	

> **COMMON EXAM TRAP**
>
> **Viewer does not mean "can read all the data."** A Viewer can query a warehouse or lakehouse through the SQL analytics endpoint and open reports, but cannot read the underlying files with Spark or OneLake APIs. If a scenario says "analysts must query tables with SQL but must not read the raw files," Viewer plus `ReadData` is the answer — not Contributor.

2.3 Item-level access control

Item permissions let you share a single item with someone who has no workspace role at all. They are additive to workspace roles: you can grant more, never less.

Permission	Meaning	Applies to
Read	See the item exists and open its metadata	All items
ReadData	Query data through the SQL analytics endpoint	Lakehouse, warehouse
ReadAll	Read the underlying OneLake files (Spark, OneLake APIs)	Lakehouse, warehouse
Build	Create new content on top of a semantic model	Semantic model
Reshare	Pass the permission on to someone else	Shared items
Write	Modify the item definition	All items
Execute	Run the item (for example a notebook or pipeline)	Executable items

Sharing a lakehouse shows exactly these switches: *Read all Apache Spark* (`ReadAll`), *Read all SQL endpoint data* (`ReadData`), and *Build reports on the default semantic model* (`Build`). Reference: Fabric permission model.

> **COMMON EXAM TRAP**
>
> **Build is not Read.** A user with Read on a semantic model can open existing reports; a user needs **Build** to create a new report, connect from Excel, or use Analyze in Excel. "Users can see the published report but cannot create their own" almost always means Build is missing.

2.4 File-level access control with OneLake security

OneLake security applies access rules at the OneLake layer, so they are enforced no matter which engine reads the data — Spark, T-SQL, Direct Lake, or the OneLake API.

You define **data access roles** on a lakehouse that specify which folders and tables a role can read, and assign Entra users or groups to those roles. Rows and columns can also be restricted at this layer, and the restriction follows the data into a Direct Lake semantic model.

- Get started: OneLake security
- The full evaluation logic: data access control model
- Role setup walkthrough: get started with data access roles

> **KNOW THE DIFFERENCE**
>
> **OneLake security vs SQL permissions.** OneLake security is enforced for *every* engine, including Spark and Direct Lake. `GRANT` / `DENY` in the SQL analytics endpoint is enforced *only* for callers coming through that endpoint — a Spark notebook with `ReadAll` bypasses it entirely. When a scenario says "regardless of the tool they use", the answer is OneLake security.

2.5 Row-level security (RLS)

RLS filters **rows**. Fabric has three separate implementations, and the exam will test whether you know which is which.

▸ RLS in a semantic model (DAX)

Define a role in Power BI Desktop (**Modeling** → **Manage roles**), write a DAX filter on a table, then assign users or groups to the role in the Fabric service.

```
-- Role "Sales rep": a rep sees only their own region.
-- Applied to the DimSalesRep table, propagating to FactSales by relationship.
[Email] = USERPRINCIPALNAME()
```

A dynamic pattern that scales to thousands of users uses a mapping table:

```
[RegionKey] =
LOOKUPVALUE(
    UserRegion[RegionKey],
    UserRegion[UserEmail], USERPRINCIPALNAME()
)
```

Details: row-level security with Power BI.

▸ RLS in a warehouse or SQL analytics endpoint (T-SQL)

A security predicate function plus a security policy, exactly as in SQL Server:

```
CREATE SCHEMA Security;
GO
CREATE FUNCTION Security.fn_SalesRepPredicate(@SalesRep AS varchar(100))
    RETURNS TABLE
    WITH SCHEMABINDING
AS
    RETURN SELECT 1 AS granted
           WHERE @SalesRep = USER_NAME()
              OR IS_ROLEMEMBER('SalesManagers') = 1;
GO
CREATE SECURITY POLICY Security.SalesRepFilter
    ADD FILTER PREDICATE Security.fn_SalesRepPredicate(SalesRep) ON dbo.FactSales
    WITH (STATE = ON);
```

Details: row-level security in Fabric Data Warehouse.

▸ RLS in an Eventhouse (KQL)

KQL databases enforce row level security with a policy on the table that rewrites the query:

```
.alter table Telemetry policy row_level_security enable
"Telemetry | where Region == current_principal_details()['UserPrincipalName']"
```

> **COMMON EXAM TRAP**
>
> Semantic-model RLS is **ignored** for a user who has Admin, Member, or Contributor on the workspace, and for anyone with Write permission on the model. Always test with **View as role**, and put report consumers in Viewer.

2.6 Column-level and object-level security

Column-level security (CLS) hides columns. In a warehouse or SQL endpoint it is plain T-SQL:

```
-- Grant only the safe columns...
GRANT SELECT ON dbo.DimCustomer(CustomerKey, CustomerName, City) TO [Analysts];
-- ...or explicitly deny the sensitive one.
DENY SELECT ON dbo.DimCustomer(NationalId) TO [Analysts];
```

Reference: column-level security.

Object-level security (OLS) hides an entire table or column *in the semantic model*, so it disappears from the field list and from metadata. It is configured with external tooling (Tabular Editor, or a TMSL script through the XMLA endpoint) because Power BI Desktop has no OLS UI.

```
// TMSL fragment: hide a table from the "Standard" role
{
  "name": "Standard",
  "tablePermissions": [
    { "name": "Payroll", "metadataPermission": "none" }
  ]
}
```

KNOW THE DIFFERENCE

CLS hides a column's values, OLS hides the object's existence. With CLS the user still sees the column name and gets an error or a blank; with OLS the table or column is not in the model at all for that role — a visual referencing it breaks rather than showing blanks.

2.7 Sensitivity labels

Information protection applies Microsoft Purview sensitivity labels to Fabric items.

- Labels are defined in the **Microsoft Purview compliance portal**, not in Fabric.
- Labels **flow downstream automatically**: label a lakehouse and the semantic models and reports built from it inherit the label.
- Labels **persist on export** to Excel, PowerPoint, PDF, and Power BI Desktop files, where they enforce encryption and access.
- Tenant settings control who can apply and change labels.

COMMON EXAM TRAP

A sensitivity label is not an access control on its own. It classifies and can encrypt exported files, but it does not stop a user who already has Read from querying the data inside Fabric. Questions that pair "must not be able to open the exported file" with "must still see the report" are testing exactly this.

2.8 Endorsement

Endorsement is how you tell users which items to trust. It affects discovery and ranking in the OneLake catalog, not permissions.

Level	Who can apply it	Meaning
Promoted	Anyone with Write on the item (workspace Contributor and above)	"My team recommends this"
Certified	Only reviewers nominated by the tenant admin	"This has been formally reviewed"
Master data	Restricted in the same way as Certified	"This is the single authoritative source for this entity"

Endorsement pairs with lineage, the OneLake catalog, and the Microsoft Purview hub for a complete governance story.

2.9 Chapter 2 self-check

1. An analyst must run T-SQL queries against a warehouse but must not read files with Spark. What do you grant?
2. Where do you configure a rule that hides the `Salary` column from everyone except HR, enforced for Spark *and* SQL *and* Direct Lake?
3. A user can open a report but cannot create a new one from the same semantic model. What is missing?
4. Who is allowed to mark a semantic model as **Certified**?
5. Why does a workspace Member still see all rows despite an RLS role being defined?

> **Answers.** 1. Workspace Viewer plus the `ReadData` item permission (not `ReadAll`). 2. OneLake security data access roles — SQL `DENY` alone would not cover Spark. 3. The **Build** permission on the semantic model. 4. Only reviewers explicitly nominated by the Fabric tenant administrator. 5. Semantic-model RLS does not apply to users with Write access on the model.

2.10 Learn more

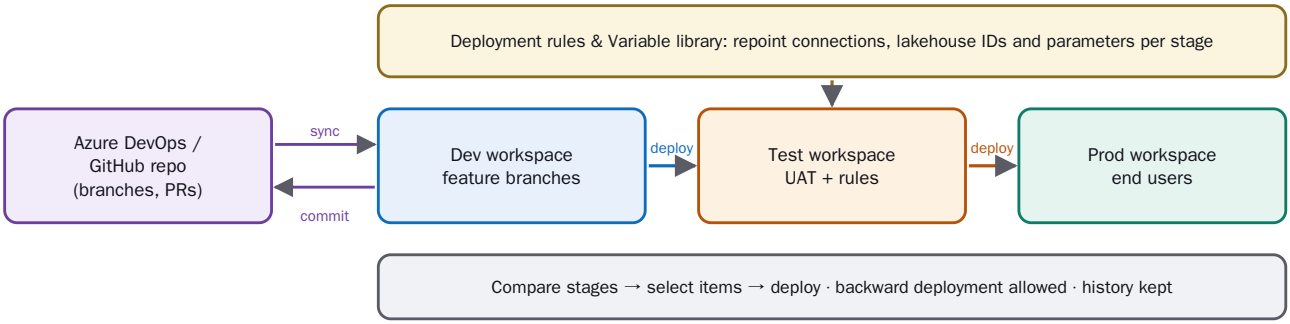
- Security in Microsoft Fabric
- Permission model
- Roles in workspaces
- OneLake security
- Row-level security in Fabric Data Warehouse
- Column-level security
- SQL granular permissions
- Information protection in Fabric
- Endorsement overview
- Training: Secure and govern analytics data in Microsoft Fabric · Secure data access in Fabric · Secure a Fabric data warehouse

---

Chapter 3 — Maintain a data analytics solution, part 2: the development lifecycle

EXAM FOCUS

Maintain the analytics development lifecycle — configure version control for a workspace · create and manage a Power BI Desktop project (.pbip) · create and configure deployment pipelines · perform impact analysis of downstream dependencies from lakehouses, warehouses, dataflows, and semantic models · deploy and manage semantic models by using the XMLA endpoint · create and update reusable assets, including Power BI template (.pbit) files, Power BI data source (.pbids) files, and shared semantic models.



Git holds the source of truth; deployment pipelines move content between stages; rules and variable libraries re-point it at the right data for each stage.

3.1 Git integration

Git integration connects a **workspace** to a **branch** in **Azure DevOps (Azure Repos)** or **GitHub**. Items are serialised into folders of text files, so you get real diffs, pull requests, and history.

How it works in practice:

1. A workspace Admin connects the workspace to an organisation, project, repository, branch, and folder.
2. **Commit** pushes workspace changes to the branch. **Update** pulls branch changes into the workspace.
3. The Source control pane shows each item as *Uncommitted*, *Modified*, or *Conflict*.
4. To work on a feature in isolation, use **Branch out to a new workspace**: Fabric creates a branch and a workspace already synced to it.

Rule	Detail
One branch per workspace	A workspace connects to exactly one branch at a time
Identity	The commit is made with the identity of the person who clicks Commit
Not everything is supported	Supported item types expand over time; unsupported items stay in the workspace but are not serialised
Conflicts	Resolved in the Source control pane, or in Git before updating

> **COMMON EXAM TRAP**
>
> Git integration syncs **item definitions, not data**. Committing a lakehouse commits its metadata and shortcuts — not the Delta files. A question describing "we need the tables and their rows in the test workspace" is describing a data-loading pipeline, not a Git update.

3.2 Power BI Desktop projects (.pbip)

A Power BI project saves a report and its semantic model as a **folder of plain-text files** instead of one opaque `.pbix`:

```
MySolution.Report/        report definition (PBIR)
MySolution.SemanticModel/ model definition in TMDL (.tmdl files)
MySolution.pbip           the small pointer file you open in Desktop
.gitignore / .platform    supporting metadata
```

Why it matters for the exam:

- It makes semantic models and reports **diffable and mergeable** — the prerequisite for meaningful code review.
- **TMDL** (Tabular Model Definition Language) is the human-readable format for the model; you can edit measures in VS Code.
- It is the recommended way to put Power BI content under source control alongside Fabric items.

> **KNOW THE DIFFERENCE**
>
> **.pbix vs .pbip vs .pbit** — `.pbix` is the classic single binary (report + model + cached data). `.pbip` is the project *folder* format for source control. `.pbit` is a *template*: report and model definition with **no data**, used to hand a starting point to someone else, who supplies parameters on open.

3.3 Deployment pipelines

Deployment pipelines move content between workspaces in ordered stages (classically Development → Test → Production; up to **10** stages are supported).

Key behaviours:

- Each stage is bound to **one workspace**. You must be an Admin of the workspace to assign it.

- Deployment **compares** stages and shows what is new, different, or missing; you choose which items to deploy.
- **Backward deployment** is allowed (Production → Test) when the target stage's workspace is empty of that item.
- Deploying does **not** move data; it moves item definitions. A warehouse deploys as an empty schema unless you also run a load.

Deployment rules re-point content per stage, so a report in Test does not read Development data:

Rule type	Typical use
Data source rule	Swap the lakehouse or SQL endpoint a semantic model reads
Parameter rule	Change a Power Query parameter such as a server or database name
Default lakehouse rule	Point a notebook at the stage's own lakehouse

For values shared by many items, a variable library holds one named variable with a different value per stage — cleaner than repeating rules item by item.

For fully automated releases, drive the same operations through the Fabric REST APIs from Azure DevOps or GitHub Actions. See CI/CD in Fabric and CI/CD best practices.

> **COMMON EXAM TRAP**
>
> Git integration and deployment pipelines solve **different** problems and are frequently confused. Git = version history, branching, code review, and recovery. Deployment pipelines = promoting a tested set of items to the next environment with stage-specific bindings. Enterprise answers usually use **both**: Git for Dev, pipelines for Dev → Test → Prod.

3.4 Impact analysis and lineage

Before you change a table, you need to know what breaks.

- **Lineage view** draws the graph of a workspace: sources → dataflows → lakehouses/warehouses → semantic models → reports and dashboards.
- **Impact analysis** starts from one item and lists every downstream dependent, across workspaces, with the number of affected reports and workspaces. It also lets you notify the contacts of those items.
- Impact analysis is available from lakehouses, warehouses, dataflows, and semantic models — exactly the four the skills outline names.

> **TIP**
>
> For programmatic dependency analysis, semantic link (`sempy`) in a notebook can list measures, relationships, and dependencies of a semantic model — useful for auditing large estates and for automated documentation.

3.5 The XMLA endpoint

The XMLA endpoint exposes a Fabric semantic model with the same protocol as Azure Analysis Services, so professional tools can connect to it.

Mode	What you can do
Read	Connect DAX Studio, Excel, SSMS, or Tabular Editor to query and profile the model
Read/Write	Deploy, alter, script, refresh, and manage partitions from external tools

Turn read/write on in the capacity settings, then connect with a workspace connection string of the form `powerbi://api.powerbi.com/v1.0/myorg/<workspace>`.

What the XMLA endpoint unlocks that Desktop cannot do:

- Deploy **metadata-only** changes (add a measure without republishing the whole model)
- Configure **object-level security** and advanced role definitions
- Manage **partitions** and trigger targeted refreshes, including `refreshType` options
- Run TMSL scripts and ALM Toolkit schema comparisons in a release pipeline

> **WATCH OUT**
> After a model is modified through the XMLA endpoint, it can no longer be downloaded as a `.pbix`. Keep the `.pbip` project in Git as your source of truth.

3.6 Reusable assets

Asset	What it contains	When to use it
.pbit (template)	Report + model definition, no data, parameters prompted on open	Distribute a standard report that each team points at its own source
.pbids (data source file)	Just the connection details	Get analysts to the right source with the right connector in one click
Shared semantic model	A published model others connect to live	One version of the truth: many reports, one model, one set of measures and RLS
Dataflow Gen2	Reusable Power Query logic with a destination	Share cleansing logic across many models
Variable library / environment	Stage-specific values, Spark libraries and config	Consistency across items and stages

A `.pbids` file is small, hand-editable JSON:

```
{
  "version": "0.1",
  "connections": [
    {
      "details": {
        "protocol": "tds",
        "address": { "server": "<workspace>.datawarehouse.fabric.microsoft.com", "database":
"SalesDW" }
      },
      "mode": "DirectQuery"
    }
  ]
}
```

3.7 Chapter 3 self-check

1. You need per-stage lakehouse bindings for ten notebooks. Rules on every item, or a variable library?
2. A developer wants two people reviewing measure changes before they reach the shared model. Which feature?
3. What is lost when you deploy a warehouse through a deployment pipeline?
4. Which tool adds object-level security to a semantic model?
5. Analysts each need the same report against their own regional database. Which asset do you publish?

> **Answers.** 1. A variable library — one variable, one value per stage, referenced by all ten. 2. Git integration with pull requests, developing in `.pbip` format so the diff is readable. 3. The data: definitions deploy, rows do not. 4. An external tool over the XMLA endpoint (Tabular Editor or a TMSL script); Desktop has no OLS UI. 5. A `.pbit` template with a parameter for the database, or a `.pbids` file pointing at their source.

3.8 Learn more

- What is lifecycle management in Fabric?
- Introduction to Git integration
- Introduction to deployment pipelines
- Deployment pipelines: get started
- CI/CD best practices
- Variable libraries
- Power BI Desktop projects
- Source control for warehouses (database projects)
- Semantic model connectivity with the XMLA endpoint
- Training: Implement CI/CD in Microsoft Fabric · Manage the semantic model development lifecycle

---

Chapter 4 — Prepare data, part 1: get data

> **EXAM FOCUS**
> **Get data** — create a data connection · discover data by using OneLake catalog and Real-Time hub · ingest or access data as needed · choose between different data stores · implement OneLake integration for Eventhouse and semantic models.

4.1 Connections and gateways

A **connection** stores where a source is and how to authenticate to it. It is created once and reused by pipelines, dataflows, and semantic models. Manage them under **Settings → Manage connections and gateways**.

Source location	What you need
Cloud SaaS or PaaS reachable from the internet	A **cloud connection** — no gateway
On-premises server, or a private network	An **on-premises data gateway** installed on a machine that can reach the source
Azure resource locked to a virtual network	A **VNet data gateway** (managed, no VM to patch)
Many users sharing one credential	A **shareable cloud connection** owned by a service account

Authentication choices you should recognise: organisational account (Microsoft Entra), service principal, account key, SAS, workspace identity, and basic. Prefer **service principals or workspace identity** for unattended jobs, so a pipeline does not break when a person leaves.

> **TIP**
> Privacy levels still matter in Power Query. If two sources have incompatible privacy levels, Power Query refuses to fold the query and may run it in a slower, memory-heavy way — or fail outright. See connector overview.

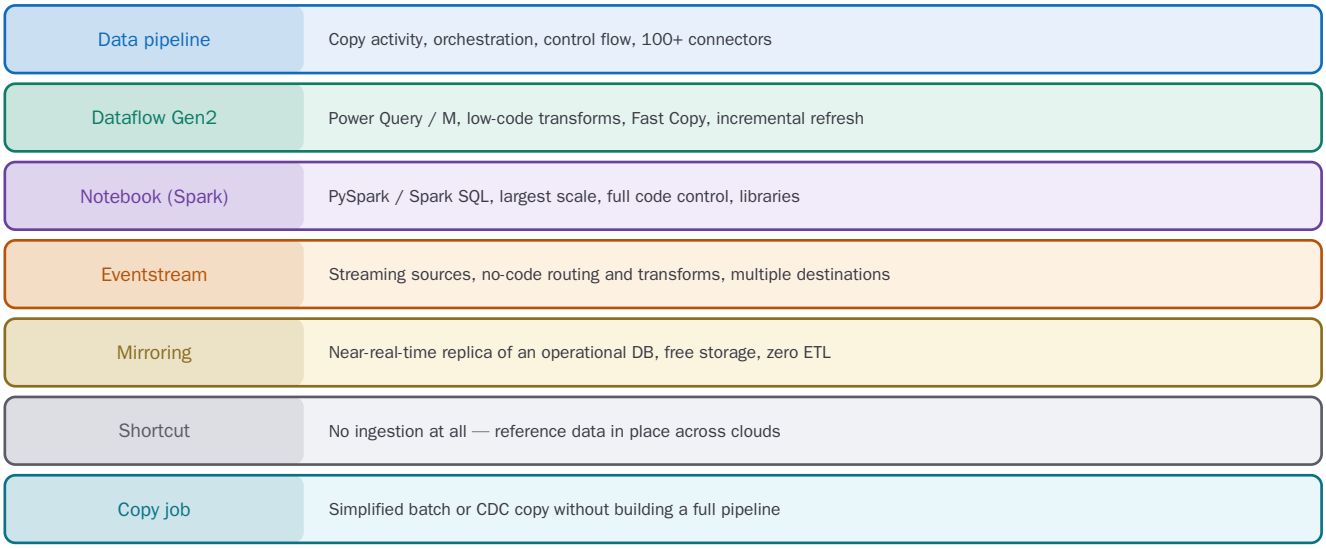
4.2 Discovering data: OneLake catalog and Real-Time hub

The OneLake catalog is the tenant-wide place to find data you are allowed to see. It shows items with their endorsement, sensitivity label, owner, and lineage, and it lets you filter by domain and item type. Governance views help admins see coverage of labels and endorsements.

The Real-Time hub is the equivalent for data **in motion**: every eventstream, Fabric event, and streaming source in the tenant, with previews and one-click actions to send a stream to an eventhouse or set an alert.

> **KNOW THE DIFFERENCE**
>
> **OneLake catalog = data at rest** (tables, files, semantic models). **Real-Time hub = data in motion** (streams and events). A question about "discover existing streams before creating a new one" points to Real-Time hub.

4.3 Ways to get data in



Seven ways to bring data into Fabric. Exam questions usually turn on volume, latency, transformation complexity, and who maintains it.

You need to...	Use
Move large volumes on a schedule with control flow, retries, and dependencies	Data pipeline with a Copy activity
Let an analyst reshape data with a low-code UI	Dataflow Gen2
Move data fast with pipeline-class throughput inside a dataflow	Fast Copy
Transform at scale, or apply custom logic and libraries	Notebook
Copy simply, batch or CDC, without building a pipeline	Copy job
Read data where it already is, across clouds	Shortcut
Replicate an operational database continuously	Mirroring
Ingest events continuously	Eventstream

Mirroring deserves special attention because it appears in both DP-600 and DP-700: it gives a **near-real-time, read-only replica** of a source database (for example Azure SQL Database, Cosmos DB, Snowflake, or PostgreSQL) landed in OneLake as Delta, with no pipeline to build and mirrored storage that does not consume your capacity's storage cost in the usual way.

4.4 OneLake integration for Eventhouse and semantic models

Two settings that turn an isolated store into part of the lake — the skills outline names them explicitly.

Eventhouse OneLake availability. Turn this on for a KQL database or table and the engine writes a Delta copy of the data into OneLake. The data stays queryable with KQL at full speed *and* becomes readable by Spark, T-SQL, shortcuts, and Direct Lake semantic models. Without it, the data is reachable only through KQL endpoints. See Eventhouse.

Semantic model OneLake integration. Turn this on for an Import-mode semantic model and its tables are written to OneLake as Delta on refresh. That lets Spark notebooks, shortcuts, and SQL read the *modelled* tables without rebuilding the logic.

> **COMMON EXAM TRAP**
>
> These two settings point in opposite directions. **Eventhouse OneLake availability** publishes KQL data *out* to the lake. **Semantic model OneLake integration** publishes model tables *out* to the lake. Neither one pulls data *in*, and neither one is how you make a Direct Lake model — for that you point a new semantic model at Delta tables that already exist in a lakehouse or warehouse.

4.5 Chapter 4 self-check

1. A nightly load from an on-premises SQL Server needs to reach a lakehouse. What is required beyond a pipeline?
2. The finance team wants Gold tables from the Sales workspace without duplicating them. What do you create?
3. A team needs Power BI reports over telemetry that lives in an eventhouse, with Direct Lake. What must be enabled?
4. Which discovery surface lists existing eventstreams?
5. A source database must appear in Fabric within seconds of a change, with no ETL to maintain. What do you choose?

> **Answers.** 1. An on-premises data gateway plus a connection that uses it. 2. A OneLake shortcut to those tables. 3. OneLake availability on the eventhouse/KQL database. 4. The Real-Time hub. 5. Mirroring.

4.6 Learn more

- What is OneLake? · OneLake shortcuts · Create a shortcut
- OneLake catalog · Real-Time hub
- Mirroring in Fabric
- Data Factory in Fabric · Copy activity · Copy job
- Dataflow Gen2 · Fast Copy
- Choose a data store

- Training: Ingest data with Microsoft Fabric · Discover and connect to data in OneLake · Choose data stores in Fabric

---

Chapter 5 — Prepare data, part 2: transform data

> **EXAM FOCUS**
> **Transform data** — create views, functions, and stored procedures · enrich data by adding new columns or tables · implement a star schema for a lakehouse or warehouse · denormalize data · aggregate data · merge or join data · identify and resolve duplicate data, missing data, or null values · convert column data types · filter data.

5.1 Choosing a transformation engine

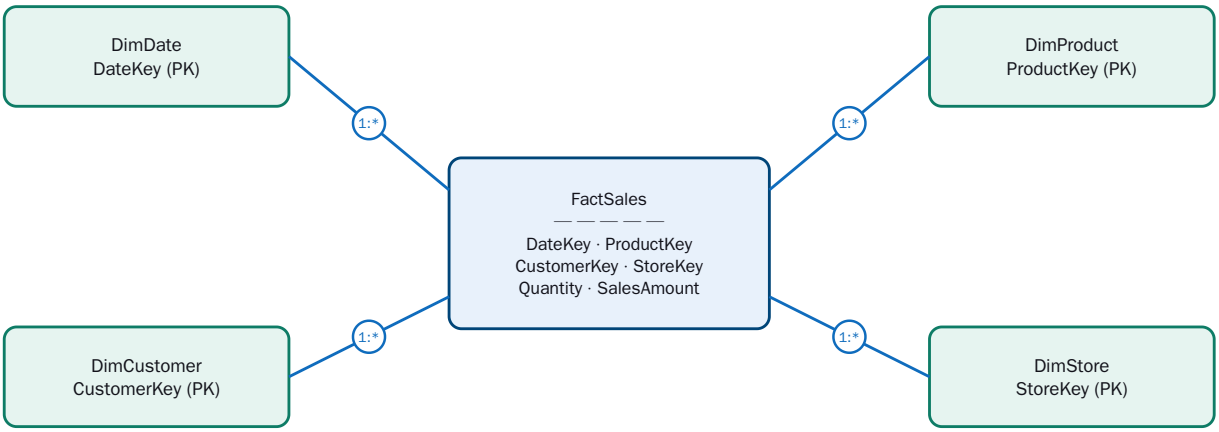
	Dataflow Gen2	Notebook (Spark)	T-SQL in a warehouse
Language	Power Query M	PySpark, Spark SQL	T-SQL
Audience	Analysts, low-code	Data engineers	SQL developers
Scale	Moderate; Fast Copy for bulk	Highest	High for set-based work
Strength	150+ connectors, visual steps, reusable	Custom logic, libraries, ML, files	Joins, MERGE, procedures, transactions
Weakness	Cost/performance at very large volume	Requires code skills, session start-up	Cannot read unstructured files
Destination	Lakehouse, warehouse, SQL DB, KQL	Any Delta table, files	Tables in the same warehouse

Official guidance: choose between Dataflow Gen2, pipelines, and Spark.

> **TIP**
> **Query folding** is the make-or-break concept for Dataflow Gen2 performance. When steps fold, Power Query translates them into a single native query at the source; when folding breaks, everything after that point is processed by the mashup engine. Check the *View native query* option and keep filters and column removal early, before any step that breaks folding.

5.2 Implementing a star schema

A star schema is the target shape for Gold. Fabric's own guidance is in dimensional modeling in Fabric, with dedicated pages for dimension tables, fact tables, and loading tables.



One-to-many, single-direction filters flowing from dimensions to the fact table

One fact table surrounded by dimensions, joined by surrogate keys. Filters flow one way: dimension to fact.

Rules worth memorising:

- **Fact tables** hold measurements plus foreign keys. Keep them narrow and long; store the key, not the description.
- **Dimension tables** hold descriptive attributes and one **surrogate key**. Keep them wide and short.
- **Grain** is declared first. "One row per order line per day" determines everything else.
- **Surrogate keys** (integer, meaningless) beat natural/business keys: they survive source system changes and make SCD Type 2 possible.
- **Conformed dimensions** (a single `DimDate`, a single `DimCustomer`) let two fact tables be analysed side by side.
- A **date dimension** is mandatory for time intelligence. Mark it as a date table in the semantic model.

Creating a dimension with a surrogate key in a warehouse:

```
CREATE TABLE dbo.DimProduct (
    ProductKey      INT           NOT NULL,   -- surrogate
    ProductId       VARCHAR(20)   NOT NULL,   -- natural/business key
    ProductName     VARCHAR(200)  NULL,
    Category        VARCHAR(100)  NULL,
    ListPrice       DECIMAL(10,2) NULL
);

-- Generate surrogate keys with a window function (Fabric Warehouse has no IDENTITY)
INSERT INTO dbo.DimProduct
SELECT ROW_NUMBER() OVER (ORDER BY s.ProductId) + COALESCE((SELECT MAX(ProductKey) FROM
dbo.DimProduct), 0),
       s.ProductId, s.ProductName, s.Category, s.ListPrice
FROM   stg.Product AS s
WHERE  NOT EXISTS (SELECT 1 FROM dbo.DimProduct d WHERE d.ProductId = s.ProductId);
```

The same idea in PySpark for a lakehouse:

```
from pyspark.sql import functions as F
from pyspark.sql.window import Window

stg = spark.read.table("bronze.product")
w   = Window.orderBy("ProductId")

dim = (stg.dropDuplicates(["ProductId"])
          .withColumn("ProductKey", F.row_number().over(w))
          .select("ProductKey", "ProductId", "ProductName", "Category", "ListPrice"))

dim.write.mode("overwrite").format("delta").saveAsTable("silver.DimProduct")
```

COMMON EXAM TRAP

Fabric Warehouse does **not** support `IDENTITY` columns, and it does not enforce `PRIMARY KEY` / `FOREIGN KEY` constraints — they can be declared as `NOT ENFORCED` for metadata and optimiser hints only. Your load process is responsible for key uniqueness and referential integrity. Expect a question where the "obvious" `IDENTITY` answer is wrong.

5.3 Denormalising, joining, and aggregating

Denormalise by folding a snowflaked hierarchy into one dimension:

```
-- Snowflake: DimProduct -> DimSubcategory -> DimCategory
-- Star: one flat DimProduct
CREATE TABLE dbo.DimProduct AS
SELECT p.ProductKey, p.ProductName,
       sc.SubcategoryName,
       c.CategoryName
FROM   stg.Product AS p
JOIN   stg.Subcategory AS sc ON sc.SubcategoryId = p.SubcategoryId
JOIN   stg.Category    AS c  ON c.CategoryId     = sc.CategoryId;
```

Fabric Warehouse supports **CTAS** (`CREATE TABLE AS SELECT`), which is the usual way to materialise a transformed table in one statement.

Aggregate to a summary table when reports only need a coarser grain:

```
CREATE TABLE dbo.AggSalesByMonth AS
SELECT d.CalendarYear, d.MonthNumber, f.ProductKey,
       SUM(f.SalesAmount) AS SalesAmount,
       SUM(f.Quantity)    AS Quantity,
       COUNT_BIG(*)       AS RowCountLines
FROM   dbo.FactSales AS f
JOIN   dbo.DimDate   AS d ON d.DateKey = f.DateKey
GROUP  BY d.CalendarYear, d.MonthNumber, f.ProductKey;
```

Join types in one line each: `INNER` keeps matches; `LEFT` keeps all of the left; `FULL` keeps everything; `ANTI` (`WHERE NOT EXISTS`) finds orphans — the standard way to detect fact rows whose dimension key is missing.

5.4 Data quality: duplicates, nulls, types

Problem	Power Query (M)	PySpark	T-SQL
Duplicates	Remove duplicates on key columns	`dropDuplicates(["Id"])`, or `row_number()` filter to keep the latest	`ROW_NUMBER() OVER (PARTITION BY Id ORDER BY ModifiedDate DESC) = 1`
Nulls	Replace values / Fill down	`fillna({"Qty": 0})`, `na.drop()`	`COALESCE(col, 0)`, `ISNULL()`
Wrong types	Change type (with locale)	`withColumn("d", F.to_date("d", "yyyy-MM-dd"))`	`TRY_CAST(col AS DATE)`
Bad rows	Keep errors / Remove errors	`filter()` to a quarantine table	`WHERE TRY_CAST(...) IS NULL` into a reject table
Filtering	Filter rows (folds to source)	`.filter()` / `.where()`	`WHERE`

Deduplicating while keeping the newest version of each record:

```
WITH ranked AS (
    SELECT *, ROW_NUMBER() OVER (PARTITION BY CustomerId ORDER BY ModifiedDate DESC) AS rn
    FROM   stg.Customer
)
SELECT * FROM ranked WHERE rn = 1;
```

> **WATCH OUT**
>
> `CAST` fails the whole batch on one bad value; `TRY_CAST` returns `NULL` instead. In a load that must not break at 3 a.m., use `TRY_CAST` and route the nulls to a quarantine table so the bad rows are visible rather than silently dropped.

5.5 Views, functions, and stored procedures

Object	Warehouse	Lakehouse SQL analytics endpoint
View	✔ Created with `CREATE VIEW`	✔ Read-only views over Delta tables
Inline table-valued function	✔	✔
Scalar function	✔	✔
Stored procedure	✔	✖ — the endpoint is read-only
`INSERT` / `UPDATE` / `DELETE` / `MERGE`	✔	✖

```
CREATE VIEW dbo.vSalesCurrentYear AS
SELECT f.*, d.CalendarYear
FROM   dbo.FactSales AS f
JOIN   dbo.DimDate   AS d ON d.DateKey = f.DateKey
WHERE  d.CalendarYear = YEAR(GETDATE());
GO

CREATE PROCEDURE dbo.usp_LoadDimProduct
AS
BEGIN
    MERGE dbo.DimProduct AS tgt
    USING stg.Product     AS src ON tgt.ProductId = src.ProductId
    WHEN MATCHED AND tgt.ListPrice <> src.ListPrice
        THEN UPDATE SET tgt.ListPrice = src.ListPrice
    WHEN NOT MATCHED BY TARGET
        THEN INSERT (ProductId, ProductName, ListPrice)
             VALUES (src.ProductId, src.ProductName, src.ListPrice);
END;
```

Call the procedure from a pipeline with a **Stored procedure activity**, or from a notebook. Check the supported surface area before you assume a T-SQL feature exists: T-SQL surface area in Fabric.

> **COMMON EXAM TRAP**
>
> The lakehouse SQL analytics endpoint is **read-only**. Any scenario requiring `MERGE`, a stored procedure, or a multi-table transaction over lakehouse tables must either move the logic into a Spark notebook (`MERGE INTO` in Spark SQL works on Delta tables) or move the Gold layer into a warehouse.

The Spark equivalent of an upsert:

```
%%sql
MERGE INTO silver.DimProduct AS tgt
USING bronze.product          AS src
   ON tgt.ProductId = src.ProductId
WHEN MATCHED AND tgt.ListPrice <> src.ListPrice THEN UPDATE SET tgt.ListPrice =
src.ListPrice
WHEN NOT MATCHED THEN INSERT *;
```

5.6 Chapter 5 self-check

1. A dataflow that ran in 3 minutes now takes 40. The only change was an added custom column early in the query. What happened?
2. You need an auto-incrementing surrogate key in a Fabric warehouse dimension. What do you use?
3. Which object cannot be created in a lakehouse SQL analytics endpoint: view, inline TVF, or stored procedure?
4. How do you keep only the most recent row per business key?
5. A date column arrives as text and 0.3% of values are malformed. How do you load the good rows without failing the batch?

> **Answers.** 1. Query folding broke at that step, so all downstream work moved to the mashup engine. 2. `ROW_NUMBER()` (or a hash/sequence pattern) — there is no `IDENTITY` in Fabric Warehouse. 3. Stored procedure. 4. `ROW_NUMBER() OVER (PARTITION BY key ORDER BY timestamp DESC) = 1`. 5. `TRY_CAST` and route the `NULL` results to a quarantine table.

5.7 Learn more

- Dimensional modeling in Fabric · dimensions · facts · load tables
- Understand star schema for Power BI
- Tables in Fabric Warehouse · Ingest data into a warehouse
- Lakehouse and Delta tables · lakehouse schemas
- Training: Transform data with Dataflows Gen2 · with notebooks · with T-SQL · Design dimensional models

---

Chapter 6 — Prepare data, part 3: query and analyse

> **EXAM FOCUS**
> **Query and analyze data** — select, filter, and aggregate data by using the Visual query editor · by using SQL · by using KQL · by using DAX.

DP-600 expects you to read and write four query surfaces. You will not be asked to write 50 lines of anything, but you will be asked to pick the correct expression out of four plausible ones.

6.1 The Visual query editor

Open a warehouse or a lakehouse SQL analytics endpoint and switch to the **Visual query** tab. It is a drag-and-drop, Power Query-style diagram: combine tables, merge, group by, filter, reduce columns — and Fabric generates the SQL for you.

- Great for analysts and for quickly prototyping a transformation.
- The result can be **saved as a view** so it becomes a reusable object.
- The generated T-SQL is visible, which makes it a good learning tool.

See visual query editor and SQL query editor.

6.2 T-SQL essentials for the exam

```
-- Top 10 products by sales in the current year, with a running total
WITH sales AS (
    SELECT p.ProductName,
           SUM(f.SalesAmount) AS SalesAmount
    FROM   dbo.FactSales   AS f
    JOIN   dbo.DimProduct  AS p ON p.ProductKey = f.ProductKey
    JOIN   dbo.DimDate     AS d ON d.DateKey    = f.DateKey
    WHERE  d.CalendarYear = YEAR(GETDATE())
    GROUP  BY p.ProductName
)
SELECT TOP (10)
       ProductName,
       SalesAmount,
       SUM(SalesAmount) OVER (ORDER BY SalesAmount DESC
                              ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS
RunningTotal,
       RANK() OVER (ORDER BY SalesAmount DESC) AS SalesRank
FROM   sales
ORDER  BY SalesAmount DESC;
```

Constructs worth having at your fingertips: CTEs, `ROW_NUMBER` / `RANK` / `DENSE_RANK`, `SUM() OVER`, `LAG` / `LEAD`, `TRY_CAST`, `COALESCE`, `STRING_AGG`, `OFFSET … FETCH`, `EXISTS` / `NOT EXISTS`, `MERGE` (warehouse only), `CTAS`.

6.3 KQL essentials for the exam

KQL reads left to right: start with a table, pipe it through operators. Reference: KQL overview.

```
// Errors per hour for the last 24 hours, worst devices first
Telemetry
| where Timestamp > ago(24h) and Level == "Error"
| summarize ErrorCount = count() by DeviceId, bin(Timestamp, 1h)
| top 20 by ErrorCount desc
| render timechart
```

Task	KQL	T-SQL analogue
Filter rows	`where`	`WHERE`
Choose columns	`project`, `project-away`	`SELECT`
Add a column	`extend`	computed column in `SELECT`
Aggregate	`summarize … by`	`GROUP BY`
Time buckets	`summarize … by bin(T, 1h)`	`DATE_BUCKET` / `DATEPART`
First N	`take`, `top … by`	`TOP`
Latest row per key	`summarize arg_max(Timestamp, *) by DeviceId`	`ROW_NUMBER() … = 1`
Distinct	`distinct`	`SELECT DISTINCT`
Variables	`let x = …;`	`DECLARE` / CTE
Visualise	`render`	—

Useful references: summarize operator · bin() · common operators tutorial.

> **COMMON EXAM TRAP**
>
> `take 10` returns *any* 10 rows with no ordering guarantee and is for sampling; `top 10 by Column desc` returns the actual top 10. Choosing `take` where the question says "the ten highest" is a classic wrong answer.

6.4 DAX essentials for the exam

```
Total Sales = SUM ( FactSales[SalesAmount] )

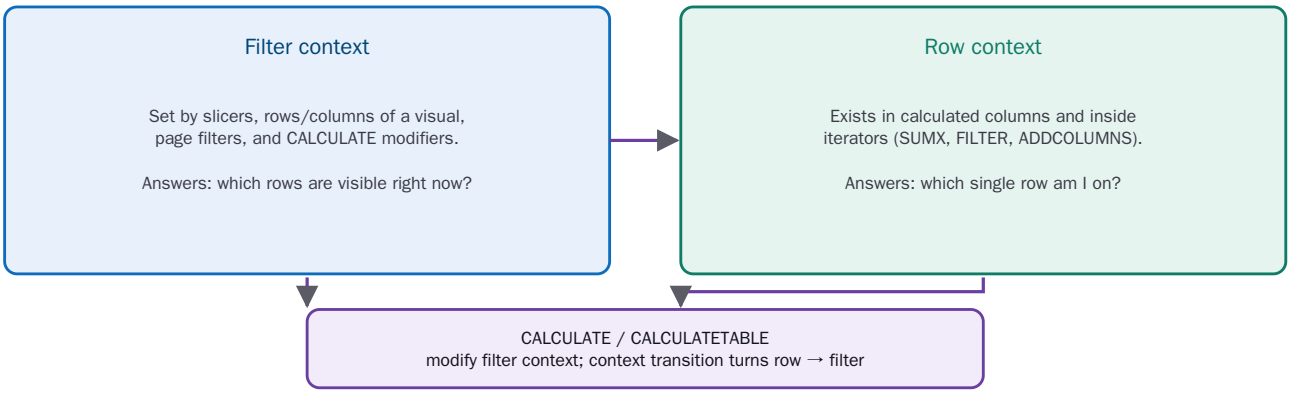
Sales LY =
CALCULATE ( [Total Sales], SAMEPERIODLASTYEAR ( DimDate[Date] ) )

YoY % =
VAR Current = [Total Sales]
VAR Prior   = [Sales LY]
RETURN DIVIDE ( Current - Prior, Prior )     -- DIVIDE handles the divide-by-zero

Top 10 Product Sales =
CALCULATE (
    [Total Sales],
    KEEPFILTERS ( TOPN ( 10, VALUES ( DimProduct[ProductName] ), [Total Sales], DESC ) )
)
```

The three things DAX questions test most often:

1. **Measure or calculated column?** A calculated column is computed at refresh, stored in the model, and has a row context. A measure is computed at query time from the current filter context. Default to measures — they cost no storage and respond to slicers.
2. **What is `CALCULATE` doing to the filter context?** It replaces filters on the same column by default; `KEEPFILTERS` intersects instead; `REMOVEFILTERS` / `ALL` clears them.
3. **Context transition.** Referencing a measure inside an iterator wraps it in an implicit `CALCULATE`, turning the current row into a filter.



Filter context and row context are different things. CALCULATE is the bridge between them.

6.5 The same question in three languages

"Total sales for 2026, by category, top 5."

```
-- T-SQL
SELECT TOP (5) c.CategoryName, SUM(f.SalesAmount) AS Sales
FROM dbo.FactSales f
JOIN dbo.DimProduct c ON c.ProductKey = f.ProductKey
JOIN dbo.DimDate d    ON d.DateKey    = f.DateKey
WHERE d.CalendarYear = 2026
GROUP BY c.CategoryName
ORDER BY Sales DESC;
```

```
// KQL
Sales
| where Year == 2026
| summarize Sales = sum(SalesAmount) by CategoryName
| top 5 by Sales desc
```

```
// DAX
Top5Categories2026 =
TOPN (
    5,
    SUMMARIZECOLUMNS (
        DimProduct[CategoryName],
        TREATAS ( { 2026 }, DimDate[CalendarYear] ),
        "Sales", [Total Sales]
    ),
    [Sales], DESC
)
```

6.6 Learn more

- Query a warehouse · Visual query editor
- KQL query language · Work with real-time data in an Eventhouse
- DAX function reference · CALCULATE · Use variables in DAX
- Training: Write DAX formulas · Create DAX calculations · Use DAX in semantic models

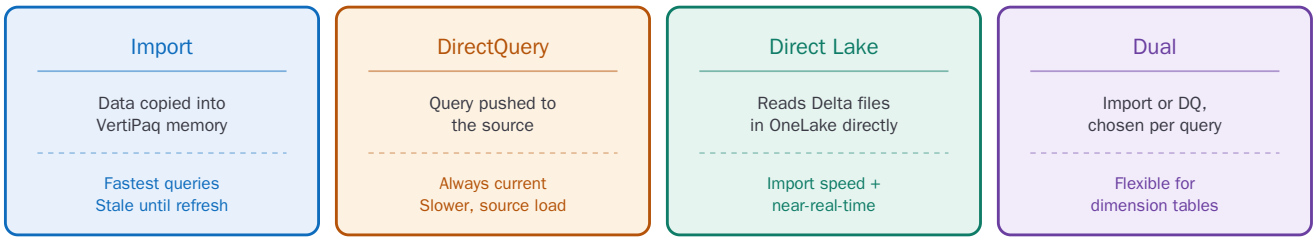
---

Chapter 7 — Implement and manage semantic models, part 1: design and build

EXAM FOCUS

Design and build semantic models — choose a storage mode · implement a star schema for a semantic model · implement relationships, such as bridge tables and many-to-many relationships · write calculations that use DAX variables and functions, such as iterators, table filtering, windowing, and information functions · implement calculation groups, dynamic format strings, and field parameters · identify use cases for and configure large semantic model storage format · design and build composite models.

7.1 Storage modes



Four storage modes. Direct Lake is the Fabric-native option and the one the exam cares about most.

	Import	DirectQuery	Direct Lake	Dual
Where data sits at query time	In VertiPaq memory	At the source	Delta files in OneLake, paged into memory	Either
Data freshness	As of last refresh	Live	Near real time after framing	Mixed
Query speed	Fastest	Depends on source	Close to Import	Fastest available
Refresh needed	Yes, full or incremental	No	Framing only (metadata)	Yes for the Import part
All DAX supported	Yes	Limited	Yes	Yes
Typical use	Most models	Huge or highly volatile sources	Lakehouse/warehouse-based models	Dimension tables in a composite model

References: storage mode · Direct Lake overview.

> **KNOW THE DIFFERENCE**
>
> **Dual is not a fourth independent mode** — it is a property you set on a table (usually a dimension) inside a composite model so the engine can satisfy a query from memory when possible and push down to the source when a DirectQuery fact table is involved. Setting dimensions to Dual is the standard fix for slow DirectQuery slicers.

7.2 Relationships

Property	Options	What to know
Cardinality	One-to-many, one-to-one, many-to-many	One-to-many from dimension to fact is the default and the goal
Cross-filter direction	Single, Both	Single is safer; Both can create ambiguity and slow queries
Active	One active path per pair of tables	Extra paths are inactive; activate them per measure with `USERELATIONSHIP`
Integrity	Assume referential integrity	Only for DirectQuery, and only when it is genuinely true

Two date relationships (order date and ship date) to one date table:

```
Sales by Ship Date =
CALCULATE ( [Total Sales], USERELATIONSHIP ( FactSales[ShipDateKey], DimDate[DateKey] ) )
```

Many-to-many situations have two solutions:

1. A **bridge table** (also called a factless fact or a junction table) containing the distinct combinations, with one-to-many relationships from each dimension into it. This is the recommended, predictable design.
2. A **many-to-many relationship** directly between two tables. Simpler to set up, but it introduces limited relationships and blank-member behaviour that surprises people.

Guidance: many-to-many relationship guidance and bi-directional filtering guidance.

> **COMMON EXAM TRAP**
>
> Turning on bi-directional filtering to "make a slicer work" is the most common self-inflicted modelling wound: it can create ambiguous filter paths, break RLS expectations, and degrade performance. The exam-correct alternatives are a bridge table, `CROSSFILTER` inside a specific measure, or `TREATAS`.

7.3 DAX the outline names explicitly

Variables — compute once, read twice, and make the intent obvious:

```
Margin % =
VAR Sales = SUM ( FactSales[SalesAmount] )
VAR Cost  = SUM ( FactSales[TotalProductCost] )
RETURN DIVIDE ( Sales - Cost, Sales )
```

Iterators (`SUMX`, `AVERAGEX`, `MAXX`, `RANKX`, `CONCATENATEX`) evaluate an expression row by row over a table and then aggregate:

```
Sales Amount = SUMX ( FactSales, FactSales[Quantity] * FactSales[UnitPrice] )
```

Table filtering — pass a table expression to `CALCULATE`:

```
High Value Sales =
CALCULATE ( [Total Sales], FILTER ( VALUES ( FactSales[OrderValue] ), FactSales[OrderValue]
> 1000 ) )
```

Windowing functions (`INDEX`, `OFFSET`, `WINDOW`, `RANK`, `ROWNUMBER`) operate over an ordered result set — much simpler than the old `EARLIER` patterns:

```
Sales vs Previous Month =
VAR Prior = CALCULATE ( [Total Sales], OFFSET ( -1, , ORDERBY ( DimDate[MonthKey] ) ) )
RETURN [Total Sales] - Prior
```

Information functions answer "what is the current state?" — `ISBLANK`, `ISFILTERED`, `ISCROSSFILTERED`, `ISINSCOPE`, `HASONEVALUE`, `SELECTEDVALUE`, `USERPRINCIPALNAME`, `USERNAME`:

```
Selected Region Label =
IF ( HASONEVALUE ( DimRegion[Region] ), SELECTEDVALUE ( DimRegion[Region] ), "All regions" )
```

7.4 Calculation groups, dynamic format strings, field parameters

These three features remove combinatorial explosion from models, and all three are named in the outline.

Calculation groups apply one set of calculation items to *any* measure. Instead of writing `Sales YTD`, `Cost YTD`, `Margin YTD`, `Sales PY`, `Cost PY`…, you write one `Time intelligence` group:

```
-- Calculation item: YTD
CALCULATE ( SELECTEDMEASURE (), DATESYTD ( DimDate[Date] ) )

-- Calculation item: PY
CALCULATE ( SELECTEDMEASURE (), SAMEPERIODLASTYEAR ( DimDate[Date] ) )

-- Calculation item: YoY %
VAR Cur = SELECTEDMEASURE ()
VAR Pri = CALCULATE ( SELECTEDMEASURE (), SAMEPERIODLASTYEAR ( DimDate[Date] ) )
RETURN DIVIDE ( Cur - Pri, Pri )
```

Reference: calculation groups.

Dynamic format strings let one measure change its own format — percentage for a ratio item, currency for an amount item, or a currency that follows the selected country:

```
-- Format string expression on a measure or calculation item
SELECTEDVALUE ( DimCurrency[FormatString], "#,##0.00" )
```

Reference: dynamic format strings.

Field parameters let the *user* choose which measure or which dimension a visual shows, from a slicer, without bookmarks:

```
Measure selection = {
    ("Sales",    NAMEOF('Measures'[Total Sales]),  0),
    ("Margin",   NAMEOF('Measures'[Margin]),       1),
    ("Quantity",NAMEOF('Measures'[Total Qty]),    2)
}
```

Reference: field parameters.

> **KNOW THE DIFFERENCE**
>
> **Calculation group vs field parameter.** A calculation group changes *how* a measure is evaluated (YTD, PY, YoY) and is authored by the modeller. A field parameter changes *which* field or measure a visual displays and is driven by the report user. They are frequently combined.

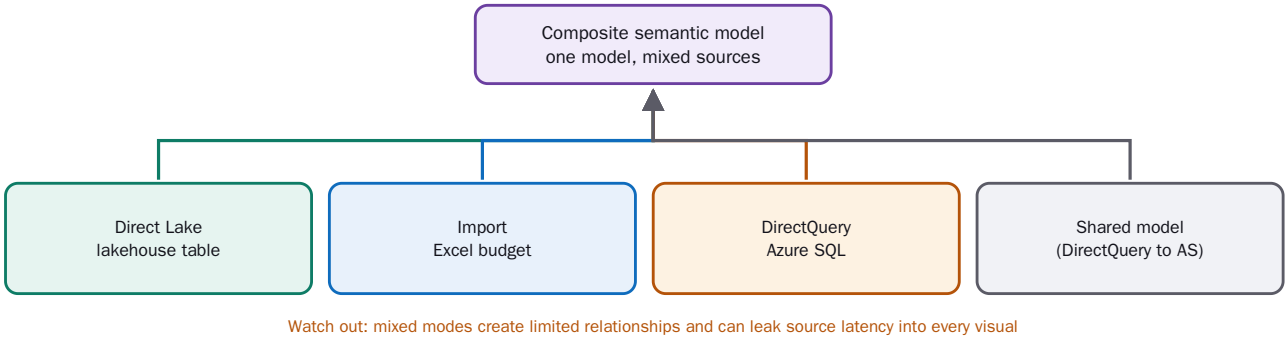
7.5 Large semantic model storage format

The large semantic model storage format stores the model in a format that supports models larger than the default limit and enables better memory management on the capacity.

- Required for models that grow past the default per-model size limit.
- Required for **Direct Lake** models, and enabled by default when Fabric creates them.
- Enables the XMLA write operations that tools use for partition-level refresh.
- It does not make a small model faster; turning it on everywhere is not the right answer.

Reduce the size before you raise the limit: remove unused columns, lower cardinality (split datetime into date + time), summarise at the grain reports actually use, and switch off **Auto date/time**. Guidance: data reduction techniques.

7.6 Composite models



A composite model mixes storage modes, and can chain onto another published semantic model.

Composite models let one model combine Import tables, DirectQuery tables, Direct Lake tables, and even a **DirectQuery connection to another semantic model**.

Use them when:

- A large fact table must stay at the source but small dimensions should be fast → DirectQuery fact + Dual dimensions.
- A central certified model must be extended locally with a team's own measures or a spreadsheet.
- Two Direct Lake sources in different workspaces must appear as one model.

Watch for: **limited relationships** (many-to-many or cross-source) that behave differently around blanks, source latency leaking into visuals, and RLS that must be reasoned about per source. Guidance: composite model guidance.

7.7 Chapter 7 self-check

1. A model needs YTD, PY, and YoY for 14 measures. What do you build?
2. Users want to switch a chart between Sales, Margin, and Quantity from a slicer. What do you build?
3. Two date columns must both filter one date dimension. How?
4. A slicer on a DirectQuery model is painfully slow, but the dimension is small. What do you change?
5. Why must a Direct Lake model use large semantic model storage format?

> **Answers.** 1. A calculation group with three calculation items. 2. A field parameter. 3. One active relationship plus an inactive one activated per measure with `USERELATIONSHIP`. 4. Set the dimension table's storage mode to Dual. 5. Direct Lake requires it; Fabric enables it automatically when the model is created.

7.8 Learn more

- Direct Lake overview · storage mode · composite models
- Star schema guidance · date tables · many-to-many
- Calculation groups · field parameters · dynamic format strings

- Large semantic model storage format · calculated tables
- Training: Design semantic models for scale · Design and manage semantic models in Fabric

---

Chapter 8 — Implement and manage semantic models, part 2: optimise at enterprise scale

> **EXAM FOCUS**
> **Optimize enterprise-scale semantic models** — implement performance improvements in queries and report visuals · improve DAX performance · configure Direct Lake, including default fallback and refresh behavior · choose between Direct Lake on OneLake and Direct Lake on SQL analytics endpoint · implement incremental refresh for semantic models.

8.1 Find the bottleneck before you fix it

Tool	What it tells you
Performance Analyzer in Desktop	Per-visual DAX query time, visual display time, and "other" — start here
DAX query view in Desktop	Run and time DAX directly, compare two rewrites of the same measure
Best Practice Analyzer (Tabular Editor)	Static rules: missing relationships, unformatted measures, unused columns
VertiPaq Analyzer metrics	Column cardinality and size — shows exactly what is making the model big
Query insights (warehouse)	Historical query text, duration, and frequency for the SQL side
Capacity Metrics app	Whether the problem is your model or a throttled capacity

Interpreting Performance Analyzer: if **DAX query** dominates, fix the model or the measure. If **visual display** dominates, the visual is rendering too many data points — reduce categories, remove unnecessary visuals, or turn off interactions. If **other** dominates, it is usually visual layout and cross-filtering between many visuals on the page.

8.2 Report and query-level improvements

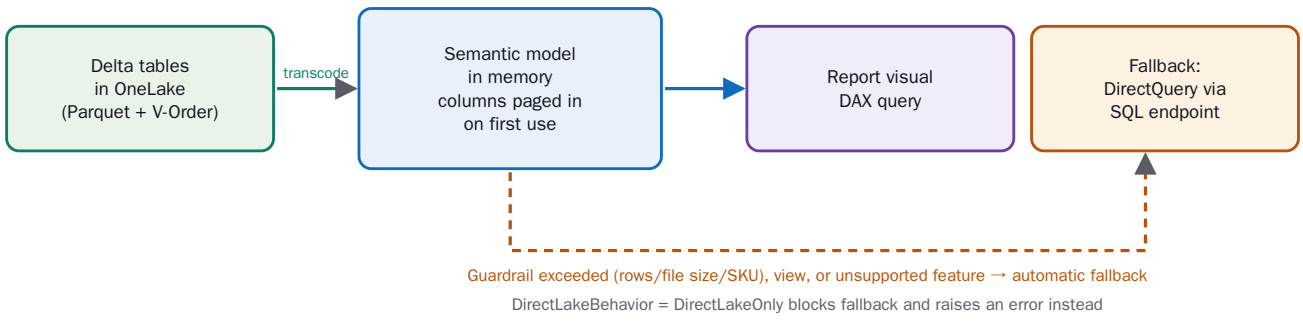
- Keep pages to roughly **8 visuals or fewer**; every visual is at least one DAX query.
- Turn off unnecessary **interactions** between visuals (Format → Edit interactions).
- Prefer **slicers with "Apply" buttons** or the filter pane over many always-on slicers.
- Avoid **bi-directional filters** and unnecessary many-to-many relationships.
- Use **aggregation tables** for high-grain fact data, with a detailed table behind them.
- Enable **query caching** for Import models on a capacity when the same report is opened repeatedly with the same filters.
- Reduce **cardinality**: split `datetime` into date and time; round floats you never analyse at full precision; delete surrogate keys that no visual uses.
- Switch off **Auto date/time** and use a proper date dimension.

8.3 DAX performance patterns

Instead of	Write	Why
`CALCULATE([Sales], FILTER(FactSales, FactSales[Amt]>100))`	`CALCULATE([Sales], FactSales[Amt]>100)`	A table filter over a large fact table materialises it; a column predicate becomes an efficient filter
`[Sales] / [Cost]`	`DIVIDE([Sales], [Cost])`	Handles blank and zero denominators without an error branch
Repeating a sub-expression three times	`VAR` it once	Evaluated once, and the intent is readable
`SUMX(FactSales, RELATED(DimProduct[Price]) * FactSales[Qty])` on 500M rows	Materialise the price on the fact at load time	Storage-engine work beats formula-engine work
`IF(ISBLANK([M]), 0, [M])` everywhere	Leave blanks blank	Blanks let the engine skip rows; forcing zeros explodes result sets
`ALL()` when you meant one column	`REMOVEFILTERS(Table[Col])`	Removing every filter is rarely what the business meant

> **KEY TAKEAWAY**
>
> The formula engine is single-threaded; the storage engine is multi-threaded and cached. Every optimisation is the same idea in different clothes: push work into the storage engine, and give it less data to touch.

8.4 Direct Lake in depth



Direct Lake reads Delta columns straight into memory on first use. Fallback to DirectQuery happens automatically unless you forbid it.

How it works. A Direct Lake table points at Delta Parquet files in OneLake. On the first query that touches a column, the engine *transcodes* that column into its in-memory VertiPaq representation and keeps it cached. Columns never queried are never loaded — which is why the first run of a report is slower than the rest.

Framing (the "refresh"). A Direct Lake model has no data refresh in the Import sense. A refresh performs **framing**: it updates model metadata to point at the current version of each Delta table and invalidates cached columns whose files changed. The next query re-pages the new data. Framing is fast because it moves metadata, not rows.

- Set the model to update automatically when source data changes, or frame on a schedule, or frame from a pipeline at the end of a load.
- Running `OPTIMIZE` on a Delta table rewrites files, so the next query must re-page those columns — expect a cold-ish first query after maintenance.

Fallback. If a query cannot be served from Direct Lake, the engine silently falls back to **DirectQuery against the SQL analytics endpoint**. Common triggers: the table exceeds the per-SKU guardrails (row counts and file sizes scale with the F SKU), the object is a SQL view rather than a table, or an unsupported construct is used.

Control it with the `DirectLakeBehavior` model property:

Value	Behaviour
`Automatic` (default)	Use Direct Lake where possible, fall back to DirectQuery silently
`DirectLakeOnly`	Never fall back — raise an error instead, so you *find out* that you exceeded a guardrail
`DirectQueryOnly`	Always use DirectQuery, useful for comparison testing

> **COMMON EXAM TRAP**
> "The report was fast yesterday and slow today, and nothing changed in the model" is the classic fallback symptom: the table grew past a guardrail and every query is now DirectQuery. Set `DirectLakeOnly` in development so a guardrail breach fails loudly instead of silently degrading in production.

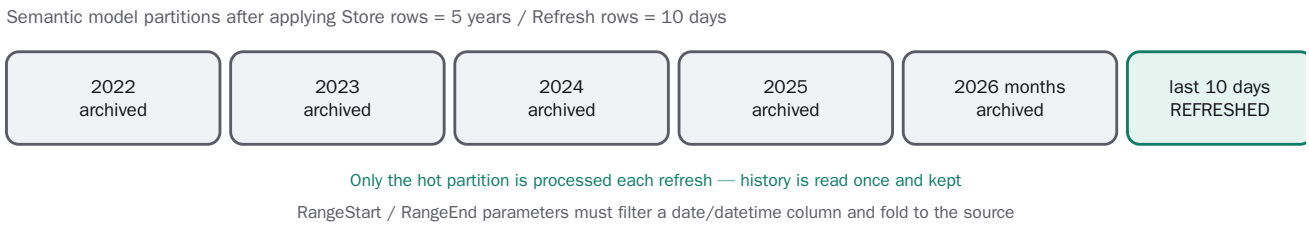
8.5 Direct Lake on OneLake vs Direct Lake on SQL analytics endpoint

	Direct Lake on SQL analytics endpoint	Direct Lake on OneLake
Points at	The lakehouse/warehouse **SQL analytics endpoint**	Delta tables in **OneLake** directly
Table sources	The tables exposed by that one endpoint	Tables from multiple lakehouses/warehouses, including via shortcuts
DirectQuery fallback	**Supported** — degrades instead of failing	**Not supported** — no SQL endpoint to fall back to
Security enforced by	SQL endpoint permissions, T-SQL RLS/CLS	OneLake security data access roles
Metadata sync	Depends on the endpoint's sync of new tables/columns	Reads the Delta log directly
Authoring	Default model, or a new model over the endpoint	Full model authoring in Power BI Desktop

> **KNOW THE DIFFERENCE**
> Pick **Direct Lake on SQL analytics endpoint** when you rely on T-SQL security objects or want automatic fallback. Pick **Direct Lake on OneLake** when the model must combine tables from several items, when you want OneLake security enforced everywhere, or when you want authoring in Desktop with no endpoint sync delay in the middle.

Further reading: Direct Lake overview · develop Direct Lake models · understand Direct Lake storage · Direct Lake in Power BI Desktop.

8.6 Incremental refresh



Incremental refresh partitions the table by date so only the recent window is reprocessed.

Incremental refresh applies to **Import** tables (and to the Import part of composite models). The setup is always the same four steps:

1. In Power Query, create two parameters named exactly `RangeStart` and `RangeEnd`, both `Date/Time`.
2. Filter the table's date column with them, using `>= RangeStart` and `< RangeEnd` so no row is counted twice.

3. In the model, set the policy: **Archive data starting *n* years before refresh date **and** Incrementally refresh data starting *m* days before refresh date**.
4. Publish and run the first refresh — that one is long, because it builds all the partitions.

Options that matter:

- **Detect data changes** — nominate a `LastModified` column so a partition is only reprocessed when something in it actually changed.
- **Only refresh complete periods** — avoids publishing a half-finished day or month.
- The source query **must fold**, otherwise every partition scans the whole source and you have made things worse.

> **COMMON EXAM TRAP**
> Incremental refresh is not a Direct Lake feature — a Direct Lake table is never "refreshed" row by row, it is framed. If a question sets up a Direct Lake model and asks how to reduce refresh time, the answer involves framing, table maintenance, and partitioned Delta writes, not `RangeStart` / `RangeEnd`.

8.7 Chapter 8 self-check

1. Performance Analyzer shows 90 ms DAX and 4,500 ms visual display. What do you fix?
2. A Direct Lake report suddenly slowed with no model change. What do you check first?
3. Which model property makes a guardrail breach raise an error instead of degrading?
4. Your incremental refresh runs as long as a full refresh. What is the likely cause?
5. You must combine Gold tables from two different lakehouses in one Direct Lake model. Which flavour?

> **Answers.** 1. The visual — too many data points or too many visuals on the page, not the measure. 2. Whether queries are falling back to DirectQuery because a guardrail was exceeded. 3. `DirectLakeBehavior = DirectLakeOnly`. 4. Query folding is broken, so each partition scans the entire source. 5. Direct Lake on OneLake.

8.8 Learn more

- Optimize semantic model performance
- Data reduction techniques
- Incremental refresh
- Performance Analyzer
- Delta Lake table optimization and V-Order
- Warehouse performance guidelines · statistics · result-set caching

---

Chapter 9 — The hands-on lab track

Reading gets you to about 500 points. The last 200 come from having actually built these things. Work through this list in order; every item is a free, guided Microsoft Learn exercise or tutorial.

#	Lab	Skill area it proves
1	Introduction to end-to-end analytics using Microsoft Fabric	Platform, workspaces, items
2	Get started with lakehouses	Lakehouse, OneLake, SQL endpoint
3	Discover and connect to data in OneLake	OneLake catalog, shortcuts
4	Use Apache Spark in Microsoft Fabric	Notebooks, DataFrames
5	Work with Delta Lake tables	Delta, time travel, optimisation
6	Ingest data with Dataflows Gen2	Dataflow Gen2, destinations
7	Orchestrate processes and data movement	Pipelines, Copy activity
8	Organize a lakehouse using medallion architecture	Bronze/Silver/Gold
9	Get started with data warehouses	Warehouse, T-SQL
10	Load data into a warehouse	CTAS, COPY INTO, stored procedures
11	Query a data warehouse	T-SQL, visual query editor
12	Design dimensional models in Fabric	Star schema, SCD
13	Get started with Real-Time Intelligence	Eventhouse, KQL
14	Secure data access in Fabric	Workspace/item/OneLake security
15	Secure a Fabric data warehouse	RLS, CLS, dynamic data masking
16	Govern analytics data in Fabric	Labels, endorsement, domains
17	Design semantic models for scale	Storage modes, star schema, Direct Lake
18	Create DAX calculations	Measures, iterators, time intelligence
19	Optimize semantic model performance	Performance Analyzer, tuning
20	Manage the semantic model development lifecycle	.pbip, Git, deployment
21	Implement CI/CD in Microsoft Fabric	Git integration, deployment pipelines
22	Monitor activities in Microsoft Fabric	Monitoring hub, refresh history

End-to-end tutorials worth a weekend: lakehouse tutorial · data warehouse tutorial · Data Factory end-to-end tutorial · Real-Time Intelligence tutorial.

Chapter 10 — Practice questions

Answer these closed-book. Full explanations follow the last question.

1. Analysts must query warehouse tables with T-SQL but must never read the underlying Parquet files with Spark. Which combination do you grant?
2. You must hide the `Salary` column from all roles except HR, enforced identically for Spark, T-SQL, and a Direct Lake model. Which feature?
3. A lakehouse table is 900 GB and grows daily. Reports must be near real time and fast. Which storage mode?
4. A Direct Lake report worked for months and is now consistently slow. Queries are hitting the SQL analytics endpoint. What happened?
5. You need YTD, PY, and YoY variants of 20 measures without writing 60 measures. What do you implement?
6. A business user must switch a chart between three different measures from a slicer. What do you implement?
7. Which two objects can you create in a lakehouse SQL analytics endpoint: view, stored procedure, inline TVF, `MERGE` statement?
8. A dataflow suddenly takes ten times longer after adding a step. What is the most likely cause?
9. You must promote a tested set of items from Test to Production and re-point them at the Production lakehouse. Which two features?
10. What exactly happens when a Direct Lake semantic model is "refreshed"?
11. Sales must be analysed by both order date and ship date against a single date dimension. How?
12. You need to know every report that breaks if you drop a column from a Gold warehouse table. Which feature?
13. Which endorsement level can only be applied by reviewers the tenant admin nominates?
14. A KQL query must return the ten devices with the most errors. Which operator?
15. An Import model refresh takes 6 hours; only the last 7 days of data ever change. What do you configure?
16. A slicer over a small dimension on a DirectQuery model is slow. What is the standard fix?
17. Which Fabric store supports multi-table transactions with T-SQL writes?
18. What does turning on OneLake availability for an eventhouse achieve?
19. Which file format lets a report and its semantic model be code-reviewed in a pull request?
20. A measure divides two measures and errors when the denominator is zero. What do you use?

▸ Answers and explanations

1. **Workspace Viewer + `ReadData` item permission.** `ReadAll` would grant Spark/OneLake file access, which the requirement forbids. Contributor would grant far too much.
2. **OneLake security data access roles.** T-SQL `DENY` only covers callers through the SQL endpoint; a Spark notebook would bypass it. OneLake security is enforced at the lake layer for every engine.

3. **Direct Lake.** Import would need a huge refresh, and DirectQuery would be slow. Direct Lake gives Import-class speed with near-real-time data through framing.
4. **The table outgrew a Direct Lake guardrail, so queries fall back to DirectQuery.** Verify with the model's fallback behaviour, then reduce the table, move to a larger SKU, or set `DirectLakeOnly` to surface the problem.
5. **A calculation group** with three calculation items using `SELECTEDMEASURE()`.
6. **A field parameter.** A calculation group changes how a measure is evaluated, not which measure is shown.
7. **View and inline TVF.** The endpoint is read-only: no stored procedures, no `MERGE`.
8. **Query folding broke at that step.** Everything downstream now runs in the mashup engine instead of the source.
9. **A deployment pipeline** (to promote) plus **deployment rules or a variable library** (to re-point the data source per stage).
10. **Framing** — metadata is updated to the current Delta version and affected cached columns are invalidated; no rows are imported. The next query pages the new data in.
11. **Two relationships**: one active on order date, one inactive on ship date, activated inside a measure with `USERELATIONSHIP`.
12. **Impact analysis** (with lineage view) from the warehouse item.
13. **Certified** (and Master data). Promoted can be applied by anyone with Write on the item.
14. `top 10 by ErrorCount desc`. `take 10` returns an arbitrary ten rows.
15. **Incremental refresh** with `RangeStart` / `RangeEnd`, archiving history and refreshing the last 7–10 days, ideally with Detect data changes.
16. **Set the dimension table's storage mode to Dual** in a composite model.
17. **Warehouse** (and Fabric SQL database for OLTP). The lakehouse SQL endpoint is read-only.
18. It writes a **Delta copy of the KQL data into OneLake**, so Spark, T-SQL, shortcuts, and Direct Lake models can read it while KQL performance is unaffected.
19. `.pbip` — the Power BI project format, with the model in TMDL text files.
20. `DIVIDE()`, which returns blank (or a supplied alternate) instead of erroring.

Chapter 11 — Exam-day checklist and quick reference

11.1 The week before

- Take the practice assessment and review **every** wrong answer against the docs, not against your memory.
- Re-read the official study guide change log — Microsoft updates the skills outline and lists exactly what moved.
- Watch the DP-600 Exam Readiness Zone session for the examiners' own framing of each skill area.
- Build one end-to-end solution from scratch in under two hours: shortcut → notebook → Gold table → Direct Lake model → report → deployment pipeline.
- Confirm your Learn profile username is correct so the credential indexes properly.

11.2 The 60-second memory dump

Write these down on the scratch pad as soon as the exam starts:

Topic	The one line
Workspace roles	Admin > Member > Contributor > Viewer; Viewer ≠ file access
Item permissions	Read · ReadData (SQL) · ReadAll (Spark/OneLake) · Build (new content)
Security scope	OneLake security = every engine; SQL GRANT = SQL endpoint only
RLS ignored for	Workspace Admin/Member/Contributor and anyone with model Write
Endorsement	Promoted (anyone with Write) · Certified / Master data (nominated reviewers)
Lakehouse vs warehouse	T-SQL writes and multi-table transactions → warehouse; files and Spark → lakehouse
SQL analytics endpoint	Read-only. No stored procedures, no `MERGE`
No `IDENTITY` in warehouse	Use `ROW_NUMBER()` for surrogate keys
Shortcut vs mirroring vs copy	Reference · replicate · duplicate
Direct Lake refresh	Framing = metadata; columns page in on first use
Fallback	Automatic → DirectQuery; `DirectLakeOnly` to fail loudly
DL on OneLake	No fallback; multiple sources; OneLake security
Incremental refresh	`RangeStart` / `RangeEnd`, must fold, Import only
Calculation group vs field parameter	How a measure is evaluated vs which field is shown
Dual storage mode	Dimensions in a composite model with a DirectQuery fact
Git vs deployment pipeline	History and review vs promotion between stages
`.pbix` / `.pbip` / `.pbit`	Binary · project folder for Git · template with no data
`take` vs `top` in KQL	Arbitrary rows vs actual top N

11.3 Reading exam questions

- Read the **last sentence first**. It tells you what is actually being asked, so you read the scenario looking for the right details.
- Look for absolute words: *must not*, *only*, *without*, *minimise administrative effort*, *regardless of the tool*. They eliminate answers.
- **Minimise effort / minimise cost** usually favours the built-in, managed option (shortcut, mirroring, deployment rules, variable library) over a custom pipeline.
- **Least-privilege** wording favours Viewer + a specific item permission over a workspace role.
- In case studies, read the requirements list before the narrative — then scan the narrative only for the facts the requirements need.
- Flag and move on. Every question is worth the same; a five-minute question costs you two easy ones.

11.4 Glossary

Term	Meaning
Capacity (F SKU)	The purchased pool of compute that workspaces run on
CU	Capacity Unit — the unit of compute consumption, smoothed over time
OneLake	The tenant-wide logical data lake, Delta Parquet, one per tenant
Shortcut	A pointer to data in another location, read in place
Mirroring	Managed near-real-time replication of an external database into OneLake
Medallion	Bronze (raw) → Silver (cleansed) → Gold (business-ready) layering
Delta Lake	The open table format Fabric uses: Parquet files plus a transaction log
V-Order	A write-time optimisation of Parquet that speeds up reads by Fabric engines
SQL analytics endpoint	The read-only T-SQL interface over a lakehouse's Delta tables
Semantic model	The tabular model (tables, relationships, measures) that reports query
Direct Lake	Storage mode that reads Delta files from OneLake into memory on demand
Framing	The metadata operation that points a Direct Lake model at the current Delta version
Transcoding	Converting a Delta column into the in-memory VertiPaq format on first use
VertiPaq	The columnar, compressed in-memory engine behind Import and Direct Lake
Calculation group	A reusable set of calculation items applied to any measure
Field parameter	A table that lets report users switch which field or measure a visual shows
XMLA endpoint	The protocol that lets external tools read from and write to a semantic model
TMDL	Tabular Model Definition Language — the text format of a model in a `.pbip` project
RLS / CLS / OLS	Row-, column-, and object-level security
Endorsement	Promoted, Certified, or Master data trust marking on an item
Domain	A grouping of workspaces by business area for federated governance
Deployment pipeline	The Fabric feature that promotes items between stage workspaces
Variable library	Named values that resolve differently per deployment stage

11.5 Where to go next

- **Renew** this certification every 12 months with the free renewal assessment.
- **Pair it with DP-700**, the Fabric Data Engineer Associate certification, to cover ingestion, streaming, and operations end to end. The DP-700 guide in this series follows the same structure.

- Keep current with the Microsoft Fabric blog and what's new in Fabric — the exam is refreshed regularly, and the skills outline carries a change log.
- Ask questions in the Fabric Community and Microsoft Q&A.

> **KEY TAKEAWAY**
>
> You do not need to memorise every menu. You need to be able to say, for any scenario: *which store, which engine, which security layer, which storage mode, and which lifecycle tool* — and to justify the choice in one sentence. That is what 700 points looks like.

Appendix A — One-page decision tables

A.1 Where should this data live?

Signal in the question	Answer
"T-SQL writes", "stored procedure", "multi-table transaction"	Warehouse
"Files", "Spark", "unstructured", "machine learning"	Lakehouse
"Telemetry", "logs", "IoT", "sub-second", "time series"	Eventhouse
"OLTP application", "point reads", "operational"	Fabric SQL database
"Do not copy the data", "data stays in S3/ADLS"	Shortcut
"Near real time replica", "no ETL to maintain"	Mirroring

A.2 Which ingestion tool?

Signal	Answer
"Orchestrate", "dependencies", "retry", "on failure"	Data pipeline
"Analyst", "low-code", "Power Query"	Dataflow Gen2
"Custom library", "very large scale", "complex logic"	Notebook
"Continuous events", "no code", "route to several destinations"	Eventstream
"Simple copy, batch or CDC, minimal setup"	Copy job

A.3 Which security control?

Signal	Answer
"Whatever tool they use"	OneLake security
"Only through SQL"	T-SQL `GRANT` / `DENY`, RLS, CLS
"Different rows per user in a report"	Semantic model RLS (DAX role)
"The column must not appear at all"	Object-level security
"The exported file must stay protected"	Sensitivity label
"Users must find the trusted version"	Endorsement + OneLake catalog

A.4 Which lifecycle tool?

Signal	Answer
"Code review", "history", "branch", "roll back"	Git integration + `.pbip`
"Promote to Test and Production"	Deployment pipeline
"Different connection per environment"	Deployment rules / variable library
"Deploy a measure without republishing"	XMLA endpoint
"What breaks if I change this?"	Impact analysis / lineage

Appendix B — Every Microsoft Learn link in this guide

Every link below already carries the Student Ambassadors Contributor ID `wt.mc_id=studentamb_657938`. Share any of them directly.

Resource	URL (includes Contributor ID)
Microsoft Certified: Fabric Analytics Engineer Associate	https://learn.microsoft.com/credentials/certifications/fabric-analytics-engineer-associate/?wt.mc_id=studentamb_657938
official DP-600 study guide	https://learn.microsoft.com/credentials/certifications/resources/study-guides/dp-600?wt.mc_id=studentamb_657938
exam duration and experience	https://learn.microsoft.com/credentials/support/exam-duration-exam-experience?wt.mc_id=studentamb_657938
exam scoring and score reports	https://learn.microsoft.com/credentials/certifications/exam-scoring-reports?wt.mc_id=studentamb_657938
exam retake policy	https://learn.microsoft.com/credentials/support/retake-policy?wt.mc_id=studentamb_657938
renew your certification	https://learn.microsoft.com/credentials/certifications/renew-your-microsoft-certification?wt.mc_id=studentamb_657938
request accommodations	https://learn.microsoft.com/credentials/certifications/request-accommodations?wt.mc_id=studentamb_657938
free DP-600 practice assessment	https://learn.microsoft.com/credentials/certifications/exams/dp-600/practice/assessment?assessment-type=practice&assessmentId=90&practice-assessment-type=certification&wt.mc_id=studentamb_657938
exam sandbox	https://learn.microsoft.com/credentials/certifications/exams/dp-600/?wt.mc_id=studentamb_657938
Get started with Microsoft Fabric	https://learn.microsoft.com/training/paths/get-started-fabric/?wt.mc_id=studentamb_657938
Implement a Lakehouse	https://learn.microsoft.com/training/paths/implement-lakehouse-microsoft-fabric/?wt.mc_id=studentamb_657938
Implement a data warehouse	https://learn.microsoft.com/training/paths/work-with-data-warehouses-using-microsoft-fabric/?wt.mc_id=studentamb_657938
Design and transform analytics data	https://learn.microsoft.com/training/paths/design-transform-analytics-data/?wt.mc_id=studentamb_657938
Secure and govern analytics data	https://learn.microsoft.com/training/paths/secure-govern-analytics-data/?wt.mc_id=studentamb_657938
Manage a Fabric environment	https://learn.microsoft.com/training/paths/manage-microsoft-fabric-environment/?wt.mc_id=studentamb_657938
Design and manage semantic models	https://learn.microsoft.com/training/paths/design-manage-semantic-models-fabric/?wt.mc_id=studentamb_657938
Exam Readiness Zone	https://learn.microsoft.com/shows/exam-readiness-zone/?wt.mc_id=studentamb_657938
Microsoft Fabric trial	https://learn.microsoft.com/fabric/fundamentals/fabric-trial?wt.mc_id=studentamb_657938
Power BI Premium Per User	https://learn.microsoft.com/power-bi/enterprise/service-premium-what-is?wt.mc_id=studentamb_657938

Resource	URL (includes Contributor ID)
Microsoft Fabric	https://learn.microsoft.com/fabric/fundamentals/microsoft-fabric-overview?wt.mc_id=studentamb_657938
admin portal	https://learn.microsoft.com/fabric/admin/admin-center?wt.mc_id=studentamb_657938
capacity settings	https://learn.microsoft.com/fabric/admin/capacity-settings?wt.mc_id=studentamb_657938
domains	https://learn.microsoft.com/fabric/governance/domains?wt.mc_id=studentamb_657938
workspaces	https://learn.microsoft.com/fabric/fundamentals/workspaces?wt.mc_id=studentamb_657938
throttling	https://learn.microsoft.com/fabric/enterprise/throttling?wt.mc_id=studentamb_657938
pause and resume	https://learn.microsoft.com/fabric/enterprise/pause-resume?wt.mc_id=studentamb_657938
Microsoft Fabric Capacity Metrics app	https://learn.microsoft.com/fabric/enterprise/metrics-app?wt.mc_id=studentamb_657938
scale a capacity	https://learn.microsoft.com/fabric/enterprise/scale-capacity?wt.mc_id=studentamb_657938
OneLake	https://learn.microsoft.com/fabric/onelake/onelake-overview?wt.mc_id=studentamb_657938
shortcut	https://learn.microsoft.com/fabric/onelake/onelake-shortcuts?wt.mc_id=studentamb_657938
Mirroring	https://learn.microsoft.com/fabric/database/mirrored-database/overview?wt.mc_id=studentamb_657938
medallion	https://learn.microsoft.com/fabric/onelake/onelake-medallion-lakehouse-architecture?wt.mc_id=studentamb_657938
choose a data store	https://learn.microsoft.com/fabric/fundamentals/decision-guide-data-store?wt.mc_id=studentamb_657938
choose between pipeline, dataflow, and Spark	https://learn.microsoft.com/fabric/fundamentals/decision-guide-pipeline-dataflow-spark?wt.mc_id=studentamb_657938
SQL analytics endpoint	https://learn.microsoft.com/fabric/data-engineering/lakehouse-sql-analytics-endpoint?wt.mc_id=studentamb_657938
create a warehouse	https://learn.microsoft.com/fabric/data-warehouse/create-warehouse?wt.mc_id=studentamb_657938
how to use notebooks	https://learn.microsoft.com/fabric/data-engineering/how-to-use-notebook?wt.mc_id=studentamb_657938
Data Factory in Fabric	https://learn.microsoft.com/fabric/data-factory/data-factory-overview?wt.mc_id=studentamb_657938

Resource	URL (includes Contributor ID)
Dataflows Gen2 overview	https://learn.microsoft.com/fabric/data-factory/dataflows-gen2-overview?wt.mc_id=studentamb_657938
Eventstream overview	https://learn.microsoft.com/fabric/real-time-intelligence/event-streams/overview?wt.mc_id=studentamb_657938
Eventhouse	https://learn.microsoft.com/fabric/real-time-intelligence/eventhouse?wt.mc_id=studentamb_657938
environments	https://learn.microsoft.com/fabric/data-engineering/create-and-use-environment?wt.mc_id=studentamb_657938
variable library	https://learn.microsoft.com/fabric/cicd/variable-library/variable-library-overview?wt.mc_id=studentamb_657938
workspace role	https://learn.microsoft.com/fabric/fundamentals/roles-workspaces?wt.mc_id=studentamb_657938
Fabric permission model	https://learn.microsoft.com/fabric/security/permission-model?wt.mc_id=studentamb_657938
OneLake security	https://learn.microsoft.com/fabric/onelake/onelake-security?wt.mc_id=studentamb_657938
OneLake security	https://learn.microsoft.com/fabric/onelake/security/get-started-security?wt.mc_id=studentamb_657938
data access control model	https://learn.microsoft.com/fabric/onelake/security/data-access-control-model?wt.mc_id=studentamb_657938
get started with data access roles	https://learn.microsoft.com/fabric/onelake/security/get-started-data-access-roles?wt.mc_id=studentamb_657938
row-level security with Power BI	https://learn.microsoft.com/power-bi/enterprise/service-admin-rls?wt.mc_id=studentamb_657938
row-level security in Fabric Data Warehouse	https://learn.microsoft.com/fabric/data-warehouse/row-level-security?wt.mc_id=studentamb_657938
column-level security	https://learn.microsoft.com/fabric/data-warehouse/column-level-security?wt.mc_id=studentamb_657938
Information protection	https://learn.microsoft.com/fabric/governance/information-protection?wt.mc_id=studentamb_657938
Endorsement	https://learn.microsoft.com/fabric/governance/endorsement-overview?wt.mc_id=studentamb_657938
lineage	https://learn.microsoft.com/fabric/governance/lineage?wt.mc_id=studentamb_657938
OneLake catalog	https://learn.microsoft.com/fabric/governance/onelake-catalog-overview?wt.mc_id=studentamb_657938
Microsoft Purview hub	https://learn.microsoft.com/fabric/governance/use-microsoft-purview-hub?wt.mc_id=studentamb_657938

Resource	URL (includes Contributor ID)
Security in Microsoft Fabric	https://learn.microsoft.com/fabric/security/security-overview?wt.mc_id=studentamb_657938
SQL granular permissions	https://learn.microsoft.com/fabric/data-warehouse/sql-granular-permissions?wt.mc_id=studentamb_657938
Secure data access in Fabric	https://learn.microsoft.com/training/modules/secure-data-access-in-fabric/?wt.mc_id=studentamb_657938
Secure a Fabric data warehouse	https://learn.microsoft.com/training/modules/secure-data-warehouse-in-microsoft-fabric/?wt.mc_id=studentamb_657938
Git integration	https://learn.microsoft.com/fabric/cicd/git-integration/intro-to-git-integration?wt.mc_id=studentamb_657938
Power BI project	https://learn.microsoft.com/power-bi/developer/projects/projects-overview?wt.mc_id=studentamb_657938
Deployment pipelines	https://learn.microsoft.com/fabric/cicd/deployment-pipelines/intro-to-deployment-pipelines?wt.mc_id=studentamb_657938
CI/CD in Fabric	https://learn.microsoft.com/fabric/cicd/cicd-overview?wt.mc_id=studentamb_657938
CI/CD best practices	https://learn.microsoft.com/fabric/cicd/best-practices-cicd?wt.mc_id=studentamb_657938
Impact analysis	https://learn.microsoft.com/fabric/governance/impact-analysis?wt.mc_id=studentamb_657938
semantic link	https://learn.microsoft.com/fabric/data-science/semantic-link-overview?wt.mc_id=studentamb_657938
XMLA endpoint	https://learn.microsoft.com/power-bi/enterprise/service-premium-connect-tools?wt.mc_id=studentamb_657938
Deployment pipelines: get started	https://learn.microsoft.com/fabric/cicd/deployment-pipelines/get-started-with-deployment-pipelines?wt.mc_id=studentamb_657938
Source control for warehouses (database projects)	https://learn.microsoft.com/fabric/data-warehouse/source-control?wt.mc_id=studentamb_657938
Implement CI/CD in Microsoft Fabric	https://learn.microsoft.com/training/modules/implement-cicd-in-fabric/?wt.mc_id=studentamb_657938
Manage the semantic model development lifecycle	https://learn.microsoft.com/training/modules/manage-semantic-model-lifecycle/?wt.mc_id=studentamb_657938
connector overview	https://learn.microsoft.com/fabric/data-factory/connector-overview?wt.mc_id=studentamb_657938
Real-Time hub	https://learn.microsoft.com/fabric/real-time-hub/real-time-hub-overview?wt.mc_id=studentamb_657938
Data pipeline	https://learn.microsoft.com/fabric/data-factory/create-first-pipeline-with-sample-data?wt.mc_id=studentamb_657938

Resource	URL (includes Contributor ID)
Copy activity	https://learn.microsoft.com/fabric/data-factory/copy-data-activity?wt.mc_id=studentamb_657938
Fast Copy	https://learn.microsoft.com/fabric/data-factory/dataflows-gen2-fast-copy?wt.mc_id=studentamb_657938
Copy job	https://learn.microsoft.com/fabric/data-factory/copy-job?wt.mc_id=studentamb_657938
Shortcut	https://learn.microsoft.com/fabric/onelake/create-onelake-shortcut?wt.mc_id=studentamb_657938
Azure SQL Database	https://learn.microsoft.com/fabric/database/mirrored-database/azure-sql-database?wt.mc_id=studentamb_657938
Ingest data with Microsoft Fabric	https://learn.microsoft.com/training/paths/ingest-data-with-microsoft-fabric/?wt.mc_id=studentamb_657938
Discover and connect to data in OneLake	https://learn.microsoft.com/training/modules/discover-data-onelake/?wt.mc_id=studentamb_657938
Choose data stores in Fabric	https://learn.microsoft.com/training/modules/choose-data-store-fabric/?wt.mc_id=studentamb_657938
dimensional modeling in Fabric	https://learn.microsoft.com/fabric/data-warehouse/dimensional-modeling-overview?wt.mc_id=studentamb_657938
dimension tables	https://learn.microsoft.com/fabric/data-warehouse/dimensional-modeling-dimension-tables?wt.mc_id=studentamb_657938
fact tables	https://learn.microsoft.com/fabric/data-warehouse/dimensional-modeling-fact-tables?wt.mc_id=studentamb_657938
loading tables	https://learn.microsoft.com/fabric/data-warehouse/dimensional-modeling-load-tables?wt.mc_id=studentamb_657938
T-SQL surface area in Fabric	https://learn.microsoft.com/fabric/data-warehouse/tsql-surface-area?wt.mc_id=studentamb_657938
Understand star schema for Power BI	https://learn.microsoft.com/power-bi/guidance/star-schema?wt.mc_id=studentamb_657938
Tables in Fabric Warehouse	https://learn.microsoft.com/fabric/data-warehouse/tables?wt.mc_id=studentamb_657938
Ingest data into a warehouse	https://learn.microsoft.com/fabric/data-warehouse/ingest-data?wt.mc_id=studentamb_657938
Lakehouse and Delta tables	https://learn.microsoft.com/fabric/data-engineering/lakehouse-and-delta-tables?wt.mc_id=studentamb_657938
lakehouse schemas	https://learn.microsoft.com/fabric/data-engineering/lakehouse-schemas?wt.mc_id=studentamb_657938
Transform data with Dataflows Gen2	https://learn.microsoft.com/training/modules/fabric-transform-data-dataflows/?wt.mc_id=studentamb_657938

Resource	URL (includes Contributor ID)
with notebooks	https://learn.microsoft.com/training/modules/fabric-transform-data-notebooks/?wt.mc_id=studentamb_657938
with T-SQL	https://learn.microsoft.com/training/modules/fabric-transform-data-tsql/?wt.mc_id=studentamb_657938
Design dimensional models	https://learn.microsoft.com/training/modules/design-dimensional-models-fabric/?wt.mc_id=studentamb_657938
visual query editor	https://learn.microsoft.com/fabric/data-warehouse/visual-query-editor?wt.mc_id=studentamb_657938
SQL query editor	https://learn.microsoft.com/fabric/data-warehouse/sql-query-editor?wt.mc_id=studentamb_657938
KQL overview	https://learn.microsoft.com/kusto/query/?view=microsoft-fabric&wt.mc_id=studentamb_657938
summarize operator	https://learn.microsoft.com/kusto/query/summarize-operator?view=microsoft-fabric&wt.mc_id=studentamb_657938
bin()	https://learn.microsoft.com/kusto/query/bin-function?view=microsoft-fabric&wt.mc_id=studentamb_657938
common operators tutorial	https://learn.microsoft.com/kusto/query/tutorials/learn-common-operators?view=microsoft-fabric&wt.mc_id=studentamb_657938
Query a warehouse	https://learn.microsoft.com/training/modules/query-data-warehouse-microsoft-fabric/?wt.mc_id=studentamb_657938
Work with real-time data in an Eventhouse	https://learn.microsoft.com/training/modules/query-data-kql-database-microsoft-fabric/?wt.mc_id=studentamb_657938
DAX function reference	https://learn.microsoft.com/dax/dax-function-reference?wt.mc_id=studentamb_657938
CALCULATE	https://learn.microsoft.com/dax/calculate-function-dax?wt.mc_id=studentamb_657938
Use variables in DAX	https://learn.microsoft.com/dax/best-practices/dax-variables?wt.mc_id=studentamb_657938
Write DAX formulas	https://learn.microsoft.com/training/modules/dax-power-bi-write-formulas/?wt.mc_id=studentamb_657938
Create DAX calculations	https://learn.microsoft.com/training/modules/dax-power-bi-create-calculations/?wt.mc_id=studentamb_657938
Use DAX in semantic models	https://learn.microsoft.com/training/paths/dax-power-bi/?wt.mc_id=studentamb_657938
storage mode	https://learn.microsoft.com/power-bi/transform-model/desktop-storage-mode?wt.mc_id=studentamb_657938
Direct Lake overview	https://learn.microsoft.com/fabric/fundamentals/direct-lake-overview?wt.mc_id=studentamb_657938

Resource	URL (includes Contributor ID)
many-to-many relationship guidance	https://learn.microsoft.com/power-bi/guidance/relationships-many-to-many?wt.mc_id=studentamb_657938
bi-directional filtering guidance	https://learn.microsoft.com/power-bi/guidance/relationships-bidirectional-filtering?wt.mc_id=studentamb_657938
calculation groups	https://learn.microsoft.com/power-bi/transform-model/calculation-groups?wt.mc_id=studentamb_657938
dynamic format strings	https://learn.microsoft.com/power-bi/create-reports/desktop-dynamic-format-strings?wt.mc_id=studentamb_657938
field parameters	https://learn.microsoft.com/power-bi/create-reports/power-bi-field-parameters?wt.mc_id=studentamb_657938
large semantic model storage format	https://learn.microsoft.com/power-bi/enterprise/service-premium-large-models?wt.mc_id=studentamb_657938
data reduction techniques	https://learn.microsoft.com/power-bi/guidance/import-modeling-data-reduction?wt.mc_id=studentamb_657938
Composite models	https://learn.microsoft.com/power-bi/transform-model/desktop-composite-models?wt.mc_id=studentamb_657938
composite model guidance	https://learn.microsoft.com/power-bi/guidance/composite-model-guidance?wt.mc_id=studentamb_657938
date tables	https://learn.microsoft.com/power-bi/guidance/model-date-tables?wt.mc_id=studentamb_657938
calculated tables	https://learn.microsoft.com/power-bi/transform-model/desktop-calculated-tables?wt.mc_id=studentamb_657938
Design semantic models for scale	https://learn.microsoft.com/training/modules/design-semantic-models-scale/?wt.mc_id=studentamb_657938
Performance Analyzer	https://learn.microsoft.com/power-bi/create-reports/desktop-performance-analyzer?wt.mc_id=studentamb_657938
Query insights	https://learn.microsoft.com/fabric/data-warehouse/query-insights?wt.mc_id=studentamb_657938
develop Direct Lake models	https://learn.microsoft.com/fabric/fundamentals/direct-lake-develop?wt.mc_id=studentamb_657938
understand Direct Lake storage	https://learn.microsoft.com/fabric/fundamentals/direct-lake-understand-storage?wt.mc_id=studentamb_657938
Direct Lake in Power BI Desktop	https://learn.microsoft.com/fabric/fundamentals/direct-lake-power-bi-desktop?wt.mc_id=studentamb_657938
Incremental refresh	https://learn.microsoft.com/power-bi/connect-data/incremental-refresh-overview?wt.mc_id=studentamb_657938
Optimize semantic model performance	https://learn.microsoft.com/training/modules/optimize-semantic-model-performance/?wt.mc_id=studentamb_657938

Resource	URL (includes Contributor ID)
Delta Lake table optimization and V-Order	https://learn.microsoft.com/fabric/data-engineering/delta-optimization-and-v-order?wt.mc_id=studentamb_657938
Warehouse performance guidelines	https://learn.microsoft.com/fabric/data-warehouse/guidelines-warehouse-performance?wt.mc_id=studentamb_657938
statistics	https://learn.microsoft.com/fabric/data-warehouse/statistics?wt.mc_id=studentamb_657938
result-set caching	https://learn.microsoft.com/fabric/data-warehouse/caching?wt.mc_id=studentamb_657938
Introduction to end-to-end analytics using Microsoft Fabric	https://learn.microsoft.com/training/modules/introduction-end-analytics-use-microsoft-fabric/?wt.mc_id=studentamb_657938
Get started with lakehouses	https://learn.microsoft.com/training/modules/get-started-lakehouses/?wt.mc_id=studentamb_657938
Use Apache Spark in Microsoft Fabric	https://learn.microsoft.com/training/modules/use-apache-spark-work-files-lakehouse/?wt.mc_id=studentamb_657938
Work with Delta Lake tables	https://learn.microsoft.com/training/modules/work-delta-lake-tables-fabric/?wt.mc_id=studentamb_657938
Ingest data with Dataflows Gen2	https://learn.microsoft.com/training/modules/use-dataflow-gen-2-fabric/?wt.mc_id=studentamb_657938
Orchestrate processes and data movement	https://learn.microsoft.com/training/modules/use-data-factory-pipelines-fabric/?wt.mc_id=studentamb_657938
Organize a lakehouse using medallion architecture	https://learn.microsoft.com/training/modules/describe-medallion-architecture/?wt.mc_id=studentamb_657938
Get started with data warehouses	https://learn.microsoft.com/training/modules/get-started-data-warehouse/?wt.mc_id=studentamb_657938
Load data into a warehouse	https://learn.microsoft.com/training/modules/load-data-into-microsoft-fabric-data-warehouse/?wt.mc_id=studentamb_657938
Get started with Real-Time Intelligence	https://learn.microsoft.com/training/modules/get-started-kusto-fabric/?wt.mc_id=studentamb_657938
Govern analytics data in Fabric	https://learn.microsoft.com/training/modules/fabric-govern-analytics-data/?wt.mc_id=studentamb_657938
Monitor activities in Microsoft Fabric	https://learn.microsoft.com/training/modules/monitor-fabric-items/?wt.mc_id=studentamb_657938
lakehouse tutorial	https://learn.microsoft.com/fabric/data-engineering/tutorial-lakehouse-introduction?wt.mc_id=studentamb_657938
data warehouse tutorial	https://learn.microsoft.com/fabric/data-warehouse/tutorial-introduction?wt.mc_id=studentamb_657938
Data Factory end-to-end tutorial	https://learn.microsoft.com/fabric/data-factory/tutorial-end-to-end-introduction?wt.mc_id=studentamb_657938

Resource	URL (includes Contributor ID)
Real-Time Intelligence tutorial	https://learn.microsoft.com/fabric/real-time-intelligence/tutorial-introduction?wt.mc_id=studentamb_657938
DP-700	https://learn.microsoft.com/credentials/certifications/fabric-data-engineer-associate/?wt.mc_id=studentamb_657938
Microsoft Fabric blog	https://www.microsoft.com/microsoft-fabric/blog/?wt.mc_id=studentamb_657938
what's new in Fabric	https://learn.microsoft.com/fabric/fundamentals/whats-new?wt.mc_id=studentamb_657938
Fabric Community	https://community.fabric.microsoft.com/?wt.mc_id=studentamb_657938
Microsoft Q&A	https://learn.microsoft.com/answers/products/?wt.mc_id=studentamb_657938